

DATA 260 Homework 5

Kavan Siddesh | Rental housing listings (DOMAIN_ID 6)

Section 0. Personal Configuration and Domain

GitHub repository	https://github.com/Kavan1804/data260-9486
SID4	9486
PORT_BASE	8486 (8000 + 9486 mod 900)
PREFIX	s9486
SEED	9486
VERIFY_SEED	269486 (260000 + 9486)
DOMAIN_ID	6 (9486 mod 8) - Rental housing listings
Hardware	Apple Mac with M1 chip and 8 GB unified memory
Local model	llama3.2:3b (Ollama 0.34.4)
Database	MySQL 9.4.0, database s9486_rel
Tagged commit hash	6c953745963b0351f744fee37eacab2b162bf46a
Submission tag	hw5

The hw5 tag points to the commit that adds this PDF and verification.json on top of the commit above; that commit has all of the code and evidence, and it is the one the self-check tested. Both collaborators (Sbnikitha, supriyaselvangesan) have access to the repository.

How HW5 extends HW4

HW5 continues the HW4 FastAPI + MySQL rental-listing app instead of starting a new one. I moved the HW4 backend and frontend to shared root-level folders with git mv (history kept; tag hw4 still has the old layout) and extended them there. The professor's DATA236_Demo5 starter was used as a reference for the related-entity pattern and the Redux Toolkit slice, not copied as a separate app.

Folder	What is in it
backend/	FastAPI + SQLAlchemy: session login (HW4), landlords + listings CRUD, relationship query
frontend/	React (Vite) client, data layer in Redux Toolkit
mcp_servers/	meals (TheMealDB) and s9486_listings (domain) MCP servers over STDIO
domain_tools/	envelope, retry/backoff, fault injection, safety rule, execute_tool, run_agent
tests/	offline test runner (no database, model or network)
reports/hw05/	RUN_LOG.txt, raw/, METRICS.md, TOOL_CONTRACTS.md, AI_USE.md, REFLECTION.md, verification.json

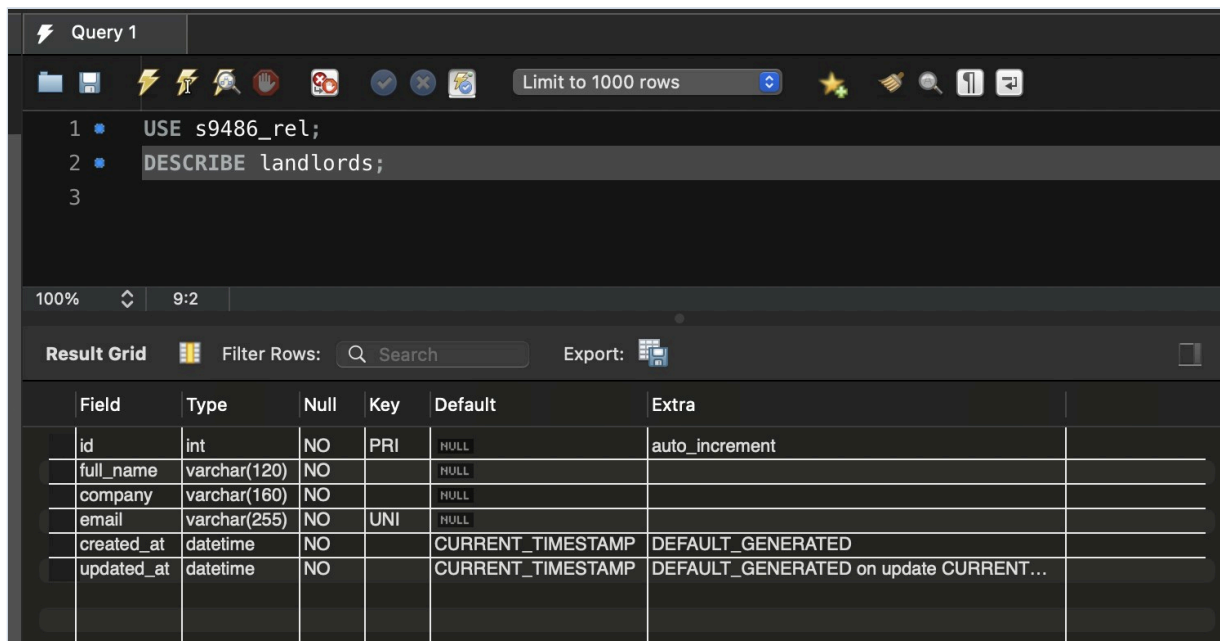
Part 1. Extending the FastAPI + MySQL Application

I. Database

The related entity is the landlord who owns a listing. One landlord has many listings, and every listing belongs to exactly one landlord.

Table	Columns and constraints
landlords (new)	id INT PK auto increment; full_name VARCHAR(120) NOT NULL (primary text); company VARCHAR(160) NOT NULL (secondary text); email VARCHAR(255) NOT NULL UNIQUE (validated, stored lower-case); created_at / updated_at DATETIME (updated_at has ON UPDATE)
listings (extended)	id INT PK; title VARCHAR(200) (primary field); address; listing_code VARCHAR(9) NOT NULL UNIQUE, format LST-12345; available_units INT NOT NULL DEFAULT 1 with CHECK >= 0; landlord_id INT NOT NULL FOREIGN KEY to landlords.id ON DELETE RESTRICT; created_at / updated_at

My database already had 5,000 HW4 listings, so I did not drop anything. The migration created the landlords table, seeded 20 landlords with SEED 9486, gave each existing listing the code LST-<id> and a seeded landlord, and only then added NOT NULL, UNIQUE, the foreign key and the CHECK. Running it again changes nothing. Deleting a landlord who still owns listings is refused twice: the API returns 409 first, and MySQL's RESTRICT foreign key blocks it as a backstop. Passwords are still only bcrypt hashes in the HW4 users table.



Field	Type	Null	Key	Default	Extra
id	int	NO	PRI		auto_increment
full_name	varchar(120)	NO			
company	varchar(160)	NO			
email	varchar(255)	NO	UNI		
created_at	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED
updated_at	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED on update CURRENT...

Figure 1. DESCRIBE landlords: unique email (UNI) and timestamps. (p1_db_landlords.png)

The screenshot shows a database query interface with a dark theme. The query editor at the top contains three lines of SQL: `USE s9486_rel;`, `DESCRIBE landlords;`, and `DESCRIBE listings;`. Below the query editor, the 'Result Grid' tab is active, displaying the schema of the 'listings' table. The table has 8 columns: Field, Type, Null, Key, Default, Extra, and an empty column. The rows represent the table's structure, including fields like 'id' (primary key, auto-increment), 'title', 'address', 'listing_code' (unique), 'available_units' (default 1), 'landlord_id' (foreign key), and two timestamp fields 'created_at' and 'updated_at' (default CURRENT_TIMESTAMP).

Field	Type	Null	Key	Default	Extra	
id	int	NO	PRI	NULL	auto_increment	
title	varchar(200)	NO		NULL		
address	varchar(255)	NO		NULL		
listing_code	varchar(9)	NO	UNI	NULL		
available_units	int	NO		1		
landlord_id	int	NO	MUL	NULL		
created_at	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED	
updated_at	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED on update CURRENT...	

Figure 2. DESCRIBE listings: unique listing_code, available_units default 1, landlord_id foreign key (MUL), timestamps. (p1_db_listings.png)

II. API

All routes use Pydantic request and response schemas and still need the HW4 session cookie, so I logged in once in Postman before these requests. List endpoints are paginated with skip and limit (1-200) and return total, skip, limit and items. PUT is a partial update, and an empty body or a null field is a 422.

Endpoint	Success	Errors handled
POST /landlords	201	409 duplicate email; 422 invalid email or missing field
GET /landlords?skip&limit	200 page	422 bad skip/limit
GET / PUT / DELETE /landlords/{id}	200	404 not found; 409 duplicate email (PUT); 409 still has listings (DELETE)
GET /landlords/{id}/listings	200 page	404 landlord not found (relationship query)
POST /listings	201	409 duplicate listing_code; 422 bad code format, negative units, unknown landlord_id
GET /listings?skip&limit	200 page	422 bad skip/limit
GET / PUT / DELETE /listings/{id}	200	404 not found; 409 / 422 as for POST (PUT)

Landlord (related entity) CRUD

The screenshot shows a REST client interface with a new request configured. The method is POST and the URL is http://localhost:8486/landlords. The request body is raw JSON: {"full_name": "Kavan Landlord", "company": "SJSU Rentals", "email": "kavan.landlord@example.com"}. The response is 201 Created, with a status bar indicating 201 ms and 315 B. The response body is JSON: {"id": 24, "full_name": "Kavan Landlord", "company": "SJSU Rentals", "email": "kavan.landlord@example.com", "created_at": "2026-10-05T12:30:04", "updated_at": "2026-10-05T12:30:04"}.

Data 260 > New Request

POST http://localhost:8486/landlords

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {"full_name": "Kavan Landlord", "company": "SJSU Rentals", "email": "kavan.landlord@example.com"}
2
```

Body Cookies 1 Headers 5 Test Results 201 Created • 201 ms • 315 B • Save Response

{ } JSON Preview Visualize

```
1 {
2   "id": 24,
3   "full_name": "Kavan Landlord",
4   "company": "SJSU Rentals",
5   "email": "kavan.landlord@example.com",
6   "created_at": "2026-10-05T12:30:04",
7   "updated_at": "2026-10-05T12:30:04"
8 }
```

Figure 3. POST /landlords creates landlord 24 (201 Created). (p1_landlord_create.png)

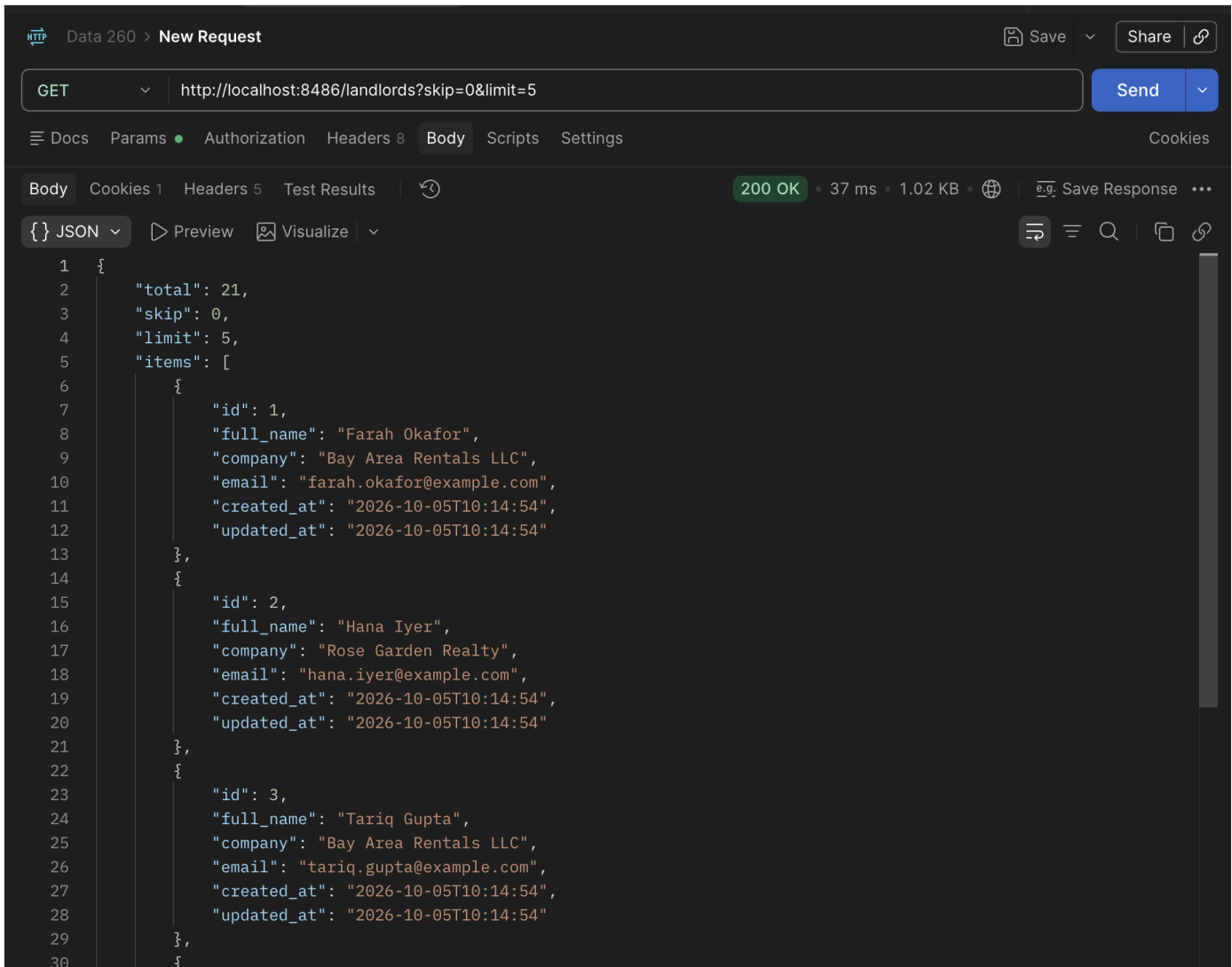


Figure 4. GET /landlords?skip=0&limit=5: paginated list (total 21 at the time). (p1_landlord_list.png)

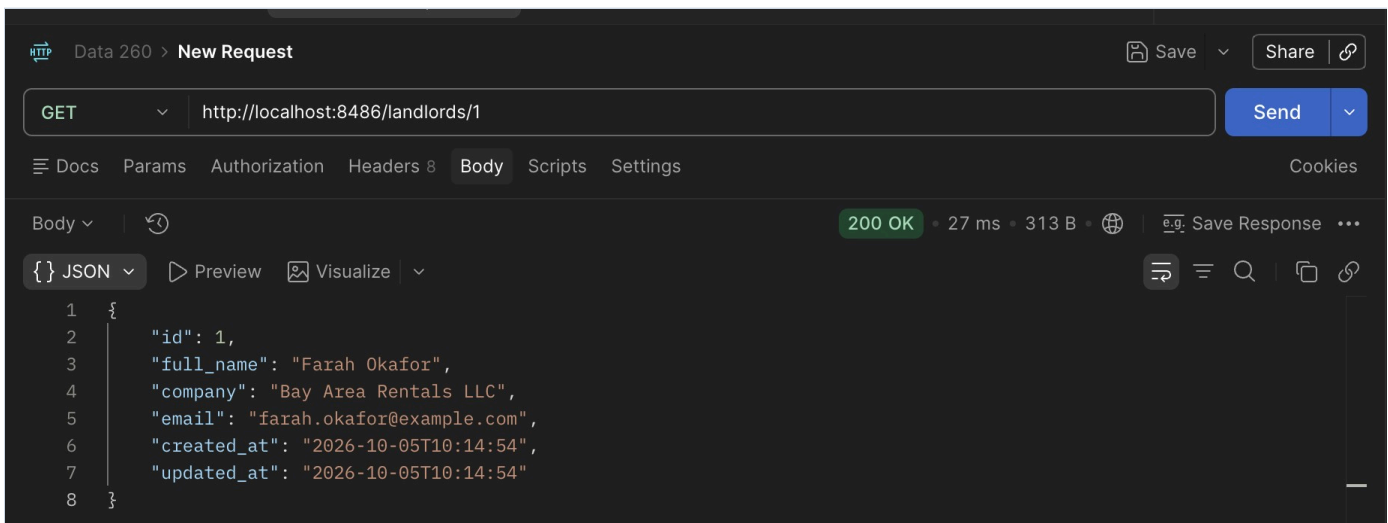


Figure 5. GET /landlords/1 returns one landlord (200). (p1_landlord_get.png)

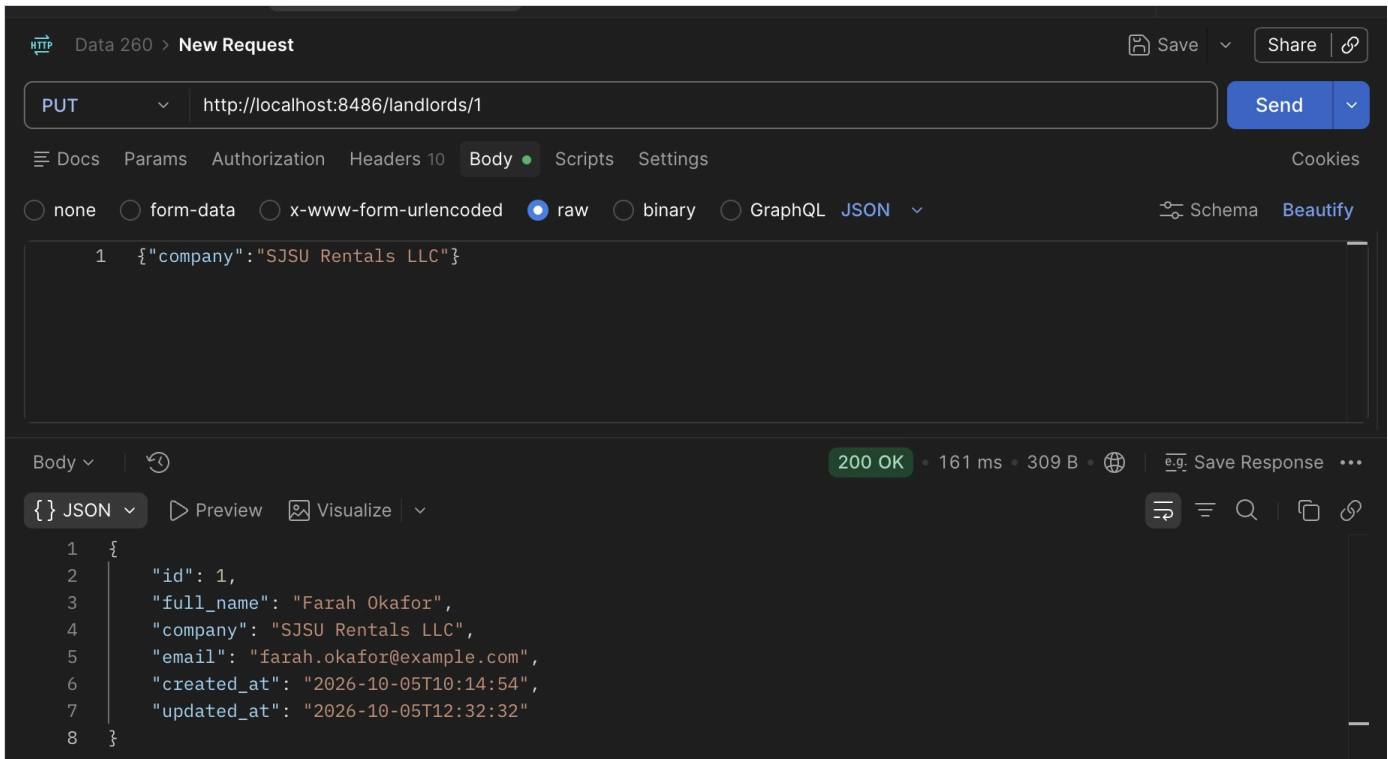


Figure 6. PUT /landlords/1 with only the company field: partial update, updated_at changes. (p1_landlord_update.png)

Listing (primary entity) CRUD

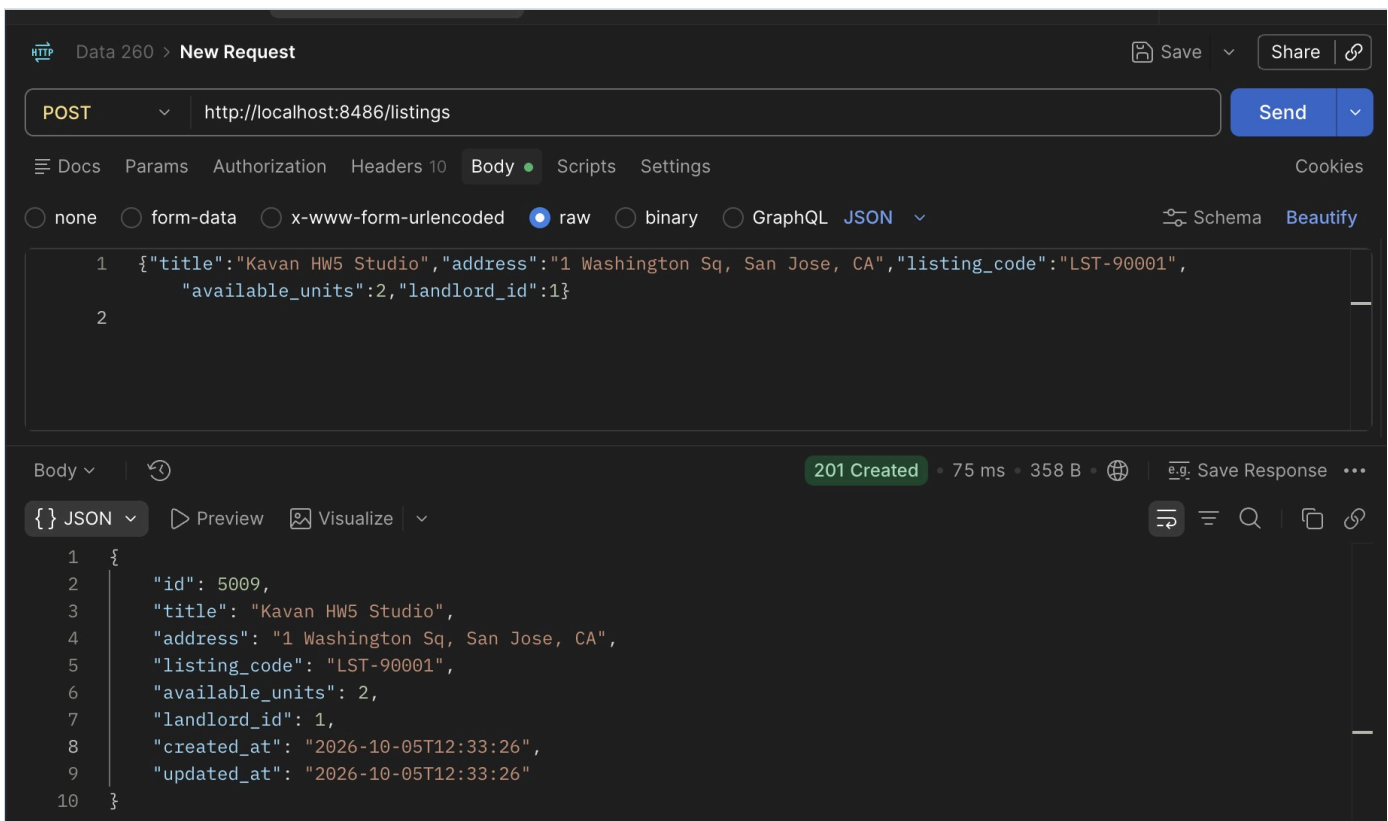


Figure 7. POST /listings with landlord_id 1 creates LST-90001 (201). (p1_listing_create.png)

```
1 {
2   "total": 5001,
3   "skip": 0,
4   "limit": 5,
5   "items": [
6     {
7       "id": 5009,
8       "title": "Kavan HW5 Studio",
9       "address": "1 Washington Sq, San Jose, CA",
10      "listing_code": "LST-90001",
11      "available_units": 2,
12      "landlord_id": 1,
13      "created_at": "2026-10-05T12:33:26",
14      "updated_at": "2026-10-05T12:33:26"
15    },
16    {
17      "id": 5002,
18      "title": "My Home",
19      "address": "2779 castor drive",
20      "listing_code": "LST-05002",
21      "available_units": 1,
22      "landlord_id": 14,
23      "created_at": "2026-10-05T10:14:54",
24      "updated_at": "2026-10-05T10:14:55"
25    },
26    {
27      "id": 4999,
28      "title": "Spacious 1BR Condo with parking",
29      "address": "7340 Market St, Mountain View, CA",
30      "listing_code": "LST-04999",
```

Figure 8. GET /listings?skip=0&limit=5: newest first, total 5001. (p1_listing_list.png)

```
1 {
2   "id": 5002,
3   "title": "My Home",
4   "address": "2779 castor drive",
5   "listing_code": "LST-05002",
6   "available_units": 1,
7   "landlord_id": 14,
8   "created_at": "2026-10-05T10:14:54",
9   "updated_at": "2026-10-05T10:14:55"
10 }
```

Figure 9. GET /listings/5002 (200). (p1_listing_get.png)

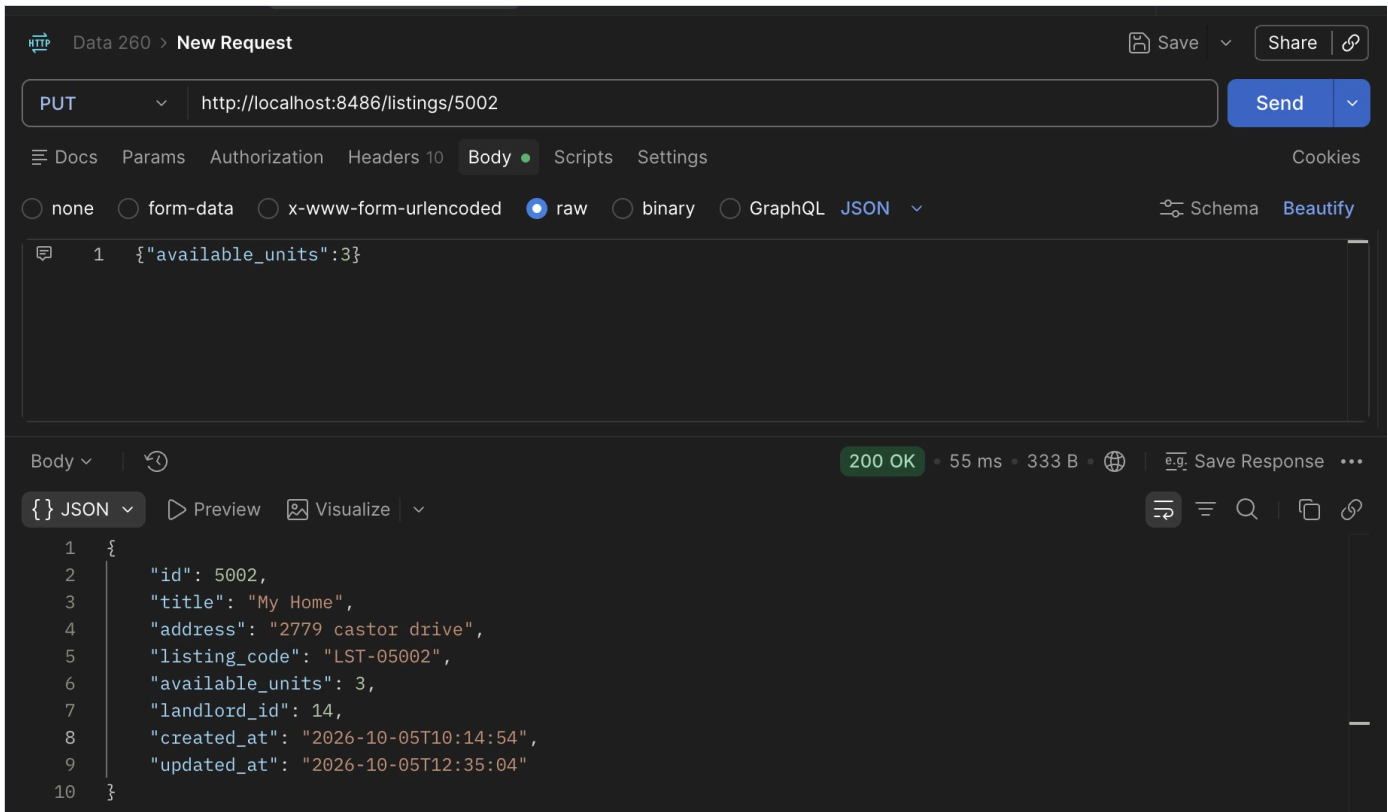


Figure 10. PUT /listings/5002 with available_units 3 (partial update, 200). (p1_listing_update.png)

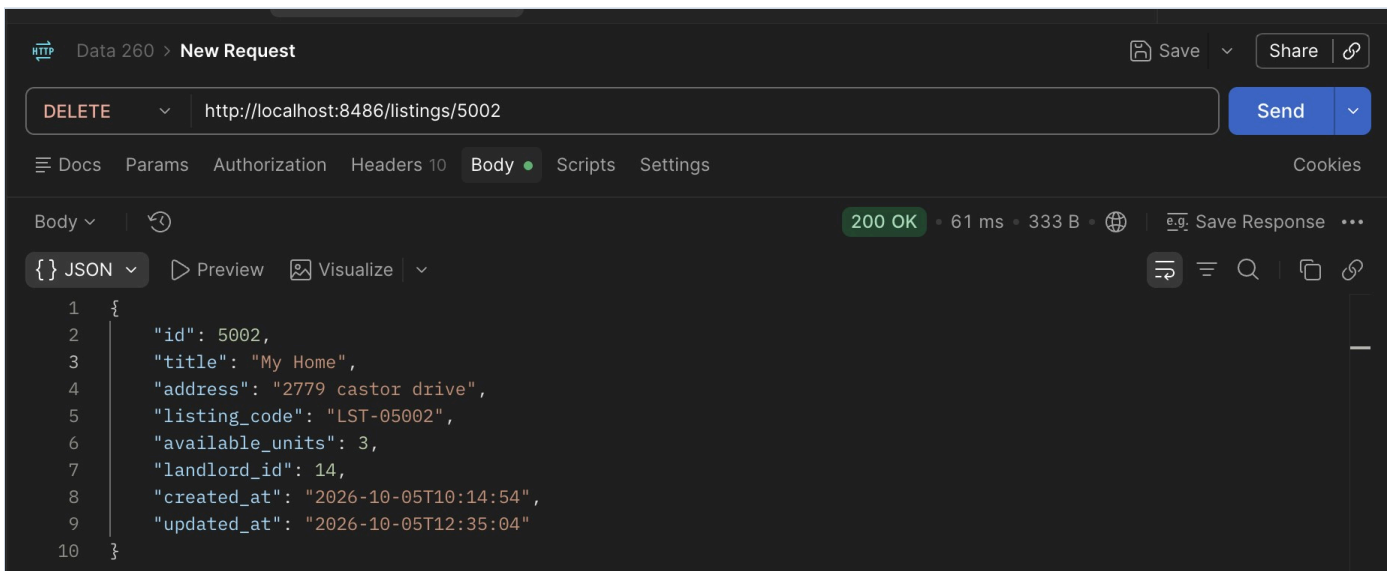
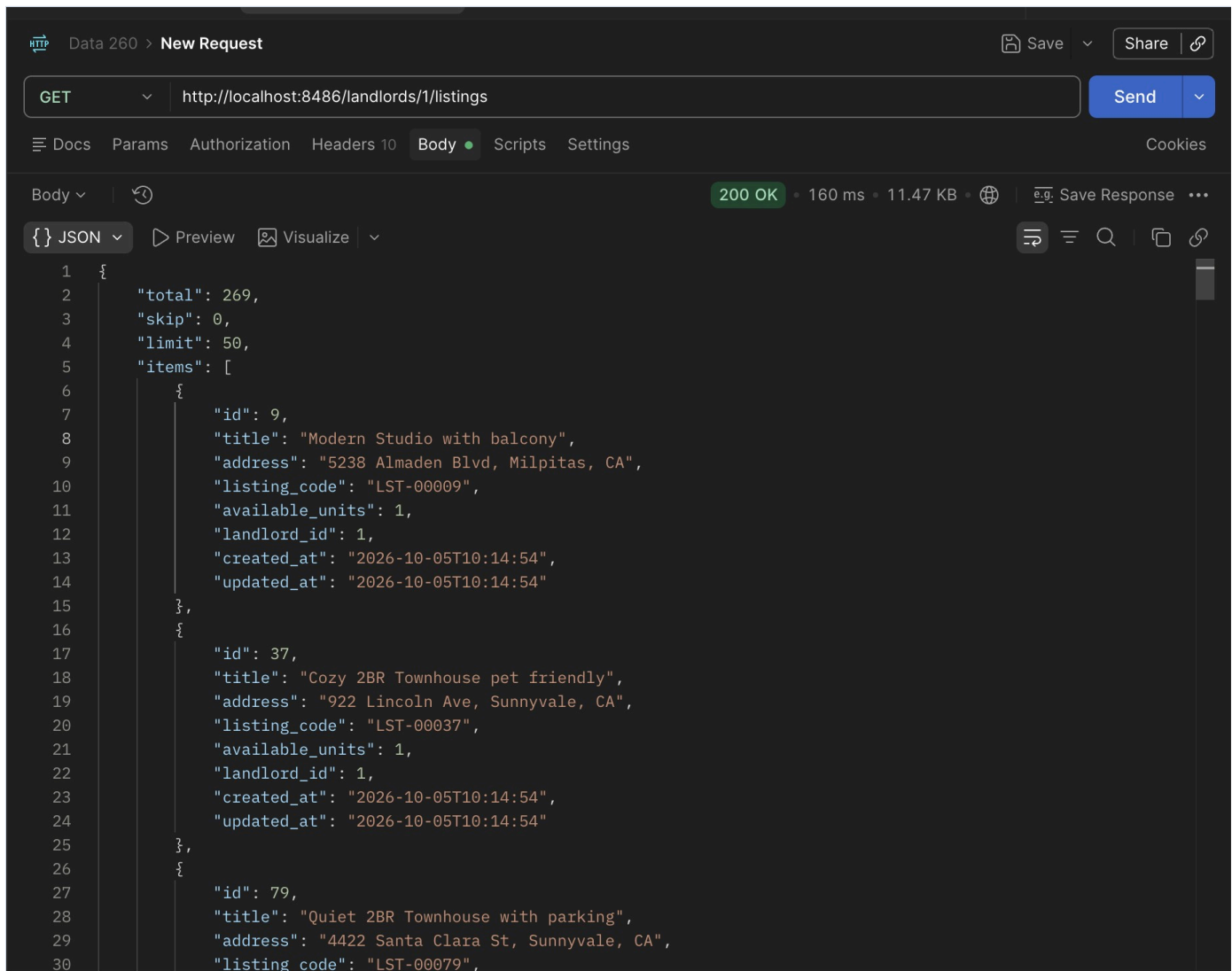


Figure 11. DELETE /listings/5002 returns the deleted listing (200). (p1_listing_delete.png)

Relationship query and error handling



The screenshot shows a REST client interface with the following details:

- Request:** GET `http://localhost:8486/landlords/1/listings`
- Response:** 200 OK, 160 ms, 11.47 KB
- Body (JSON):**

```
{
  "total": 269,
  "skip": 0,
  "limit": 50,
  "items": [
    {
      "id": 9,
      "title": "Modern Studio with balcony",
      "address": "5238 Almaden Blvd, Milpitas, CA",
      "listing_code": "LST-00009",
      "available_units": 1,
      "landlord_id": 1,
      "created_at": "2026-10-05T10:14:54",
      "updated_at": "2026-10-05T10:14:54"
    },
    {
      "id": 37,
      "title": "Cozy 2BR Townhouse pet friendly",
      "address": "922 Lincoln Ave, Sunnyvale, CA",
      "listing_code": "LST-00037",
      "available_units": 1,
      "landlord_id": 1,
      "created_at": "2026-10-05T10:14:54",
      "updated_at": "2026-10-05T10:14:54"
    },
    {
      "id": 79,
      "title": "Quiet 2BR Townhouse with parking",
      "address": "4422 Santa Clara St, Sunnyvale, CA",
      "listing_code": "LST-00079",
      "available_units": 1,
      "landlord_id": 1,
      "created_at": "2026-10-05T10:14:54",
      "updated_at": "2026-10-05T10:14:54"
    }
  ]
}
```

Figure 12. GET `/landlords/1/listings`: all listings of landlord 1 (total 269), paginated. (p1_relationship.png)

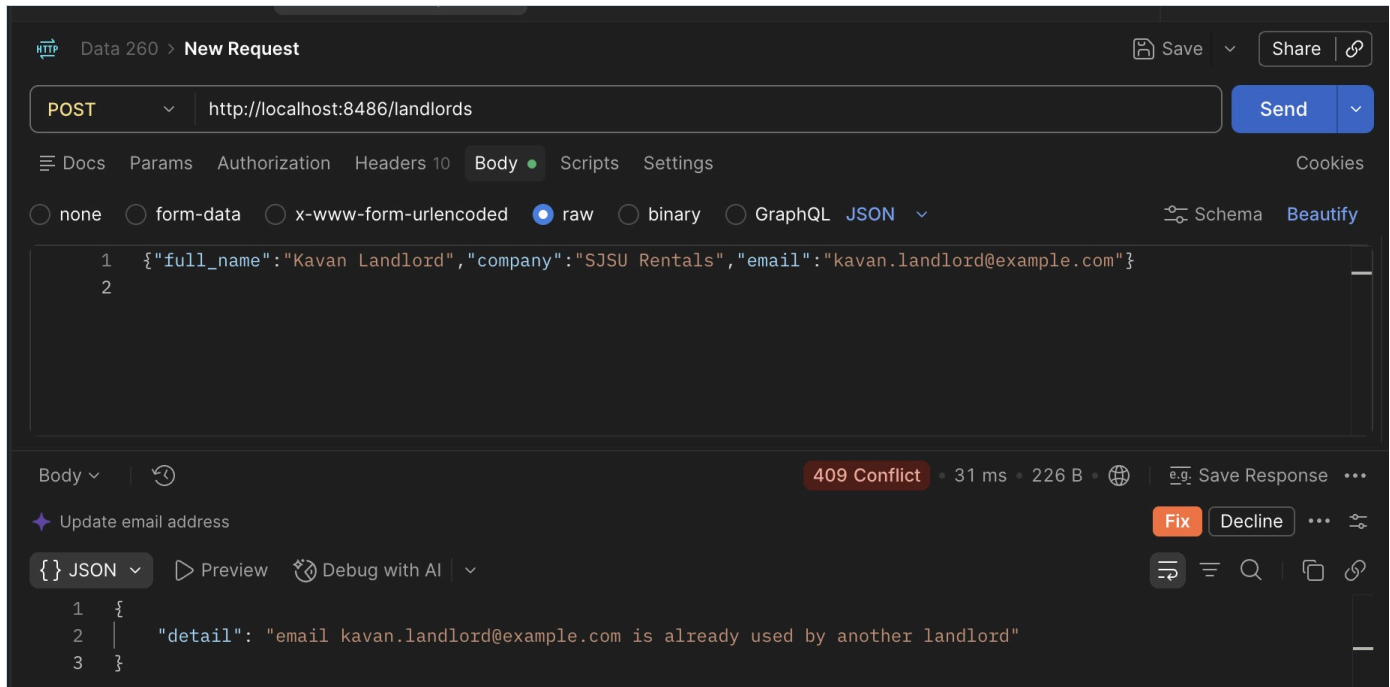


Figure 13. Unique-constraint error: the same landlord email again returns 409 Conflict.
(p1_err_duplicate_409.png)

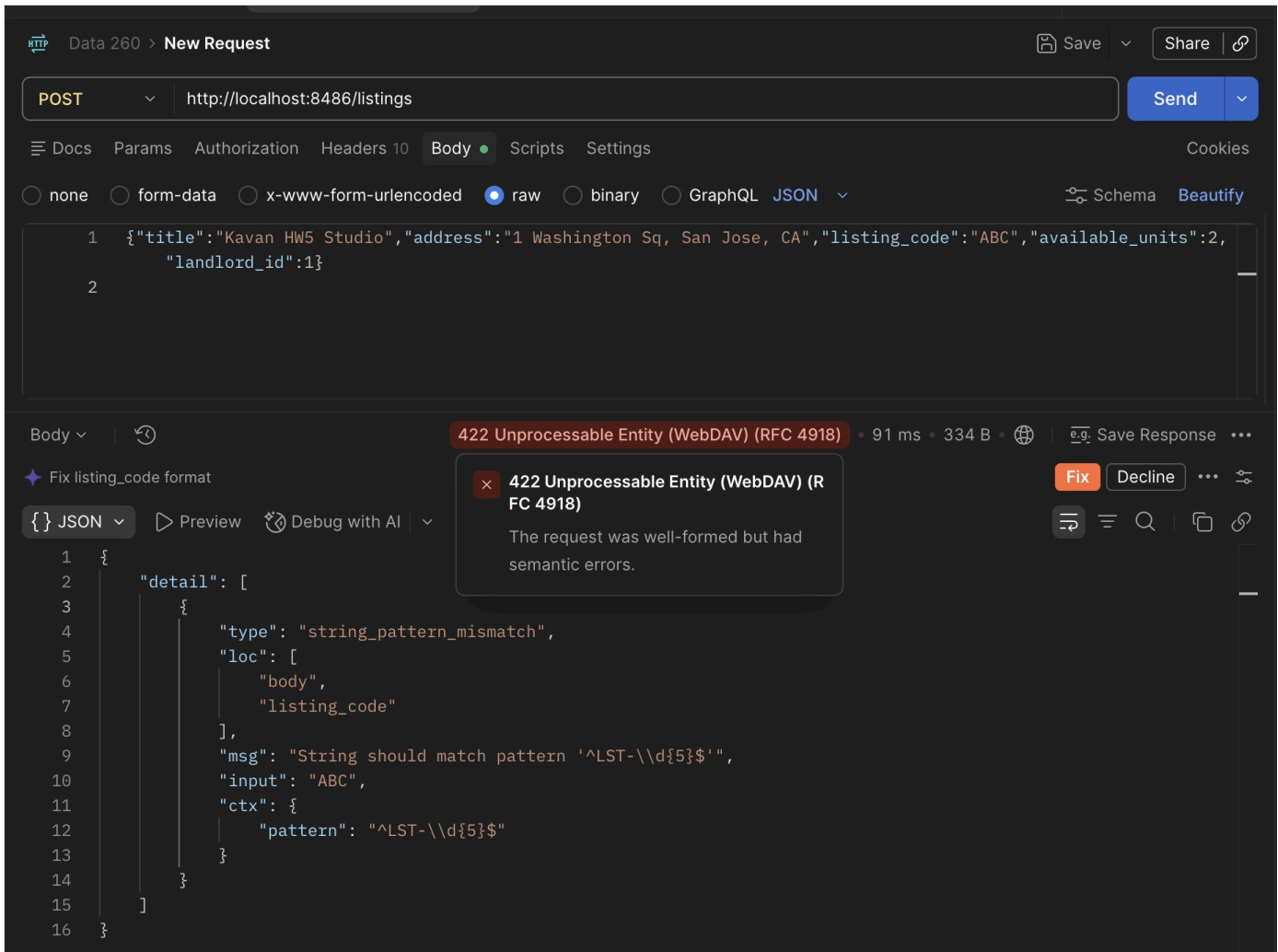


Figure 14. Unique-field format validation: listing_code "ABC" fails the LST-12345 pattern (422).
(p1_err_format_422.png)

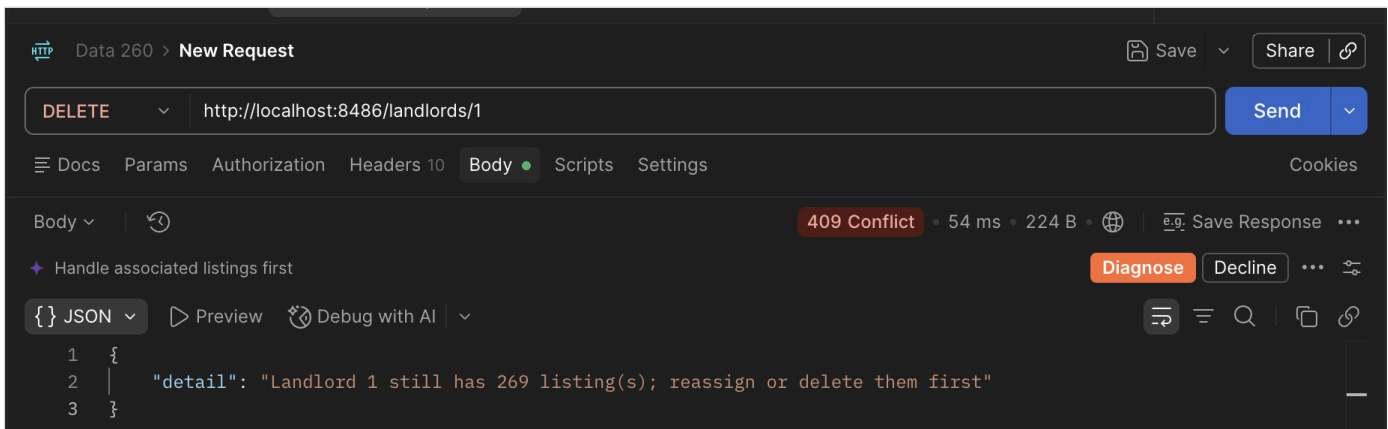


Figure 15. Relationship error: landlord 1 still owns 269 listings, so DELETE is refused (409).
(p1_err_restrict_409.png)

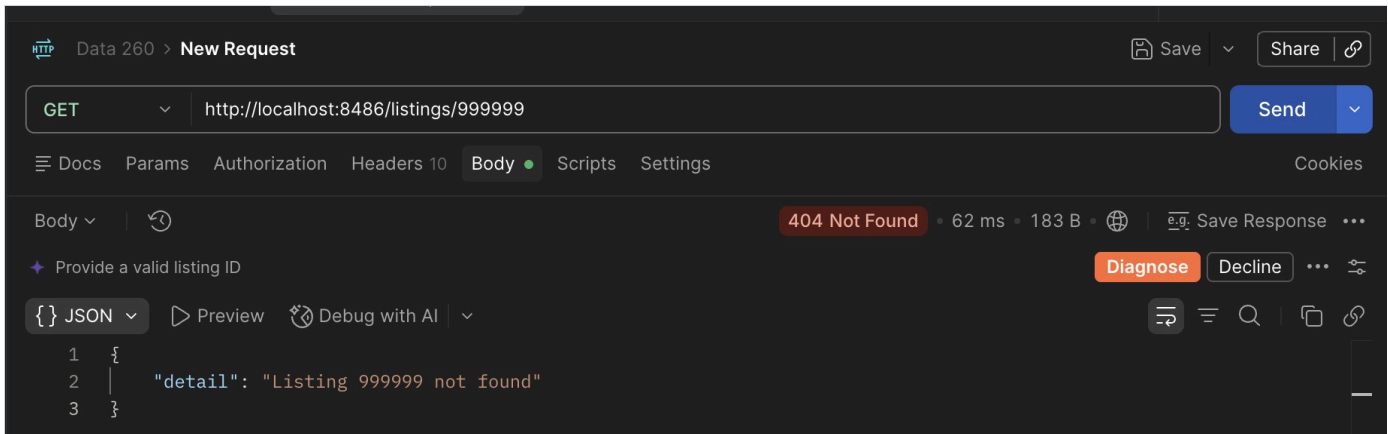


Figure 16. Not found: GET /listings/999999 returns 404. (p1_err_notfound_404.png)

III. Redux Client

I moved the React data layer from `useState` in App to Redux Toolkit. The store has a listings slice and a small landlords slice (for the landlord dropdown and names). The listings slice has four Axios async thunks: `fetch` (GET /listings), `create` (POST), `update` (PUT /listings/{id}) and `delete` (DELETE /listings/{id}). Each thunk rejects with the FastAPI error message, and the slice stores loading/error state plus a success notice. Home reads only from the store, so the list updates by itself after create, update and delete. Each screenshot shows the thunk and reducer code in VS Code next to the result in the browser.

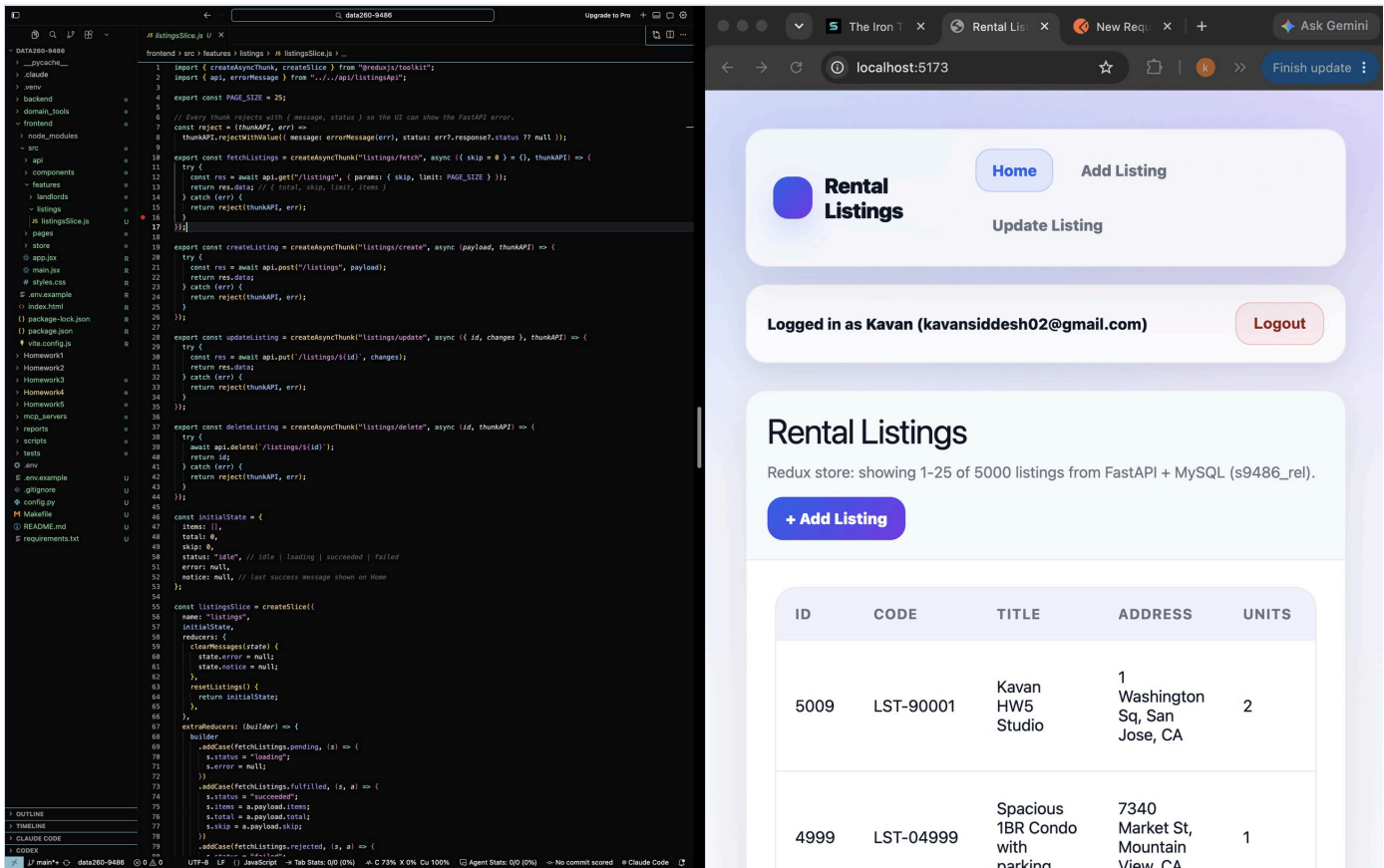


Figure 17. Home: `fetchListings` thunk and its pending/fulfilled/rejected cases (left); Home rendering the store ("showing 1-25 of 5000") (right). (p1_redux_home.png)

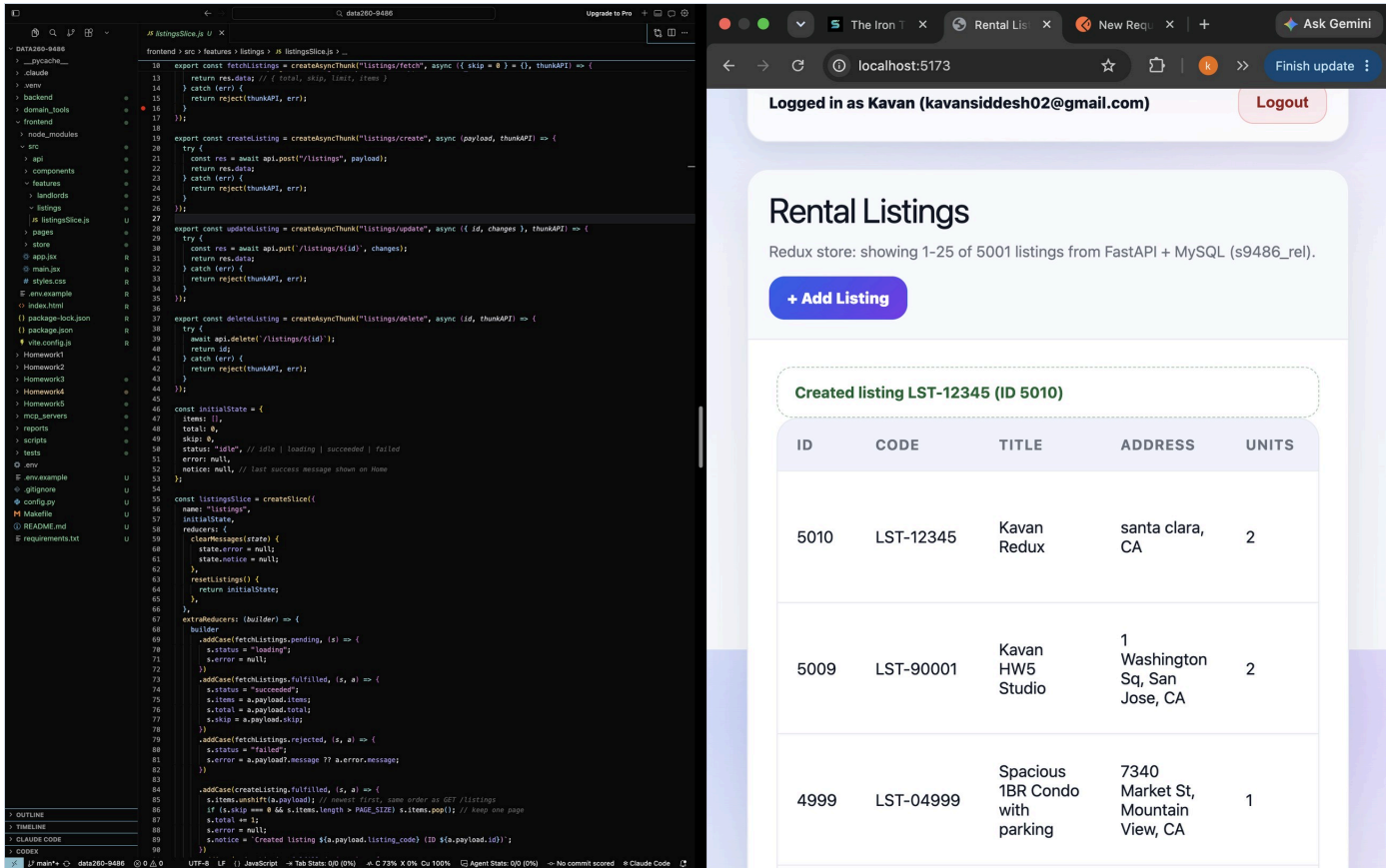


Figure 18. Create: createListing thunk and createListing.fulfilled, which puts the new row first; Home shows "Created listing LST-12345 (ID 5010)". (p1_redux_create.png)

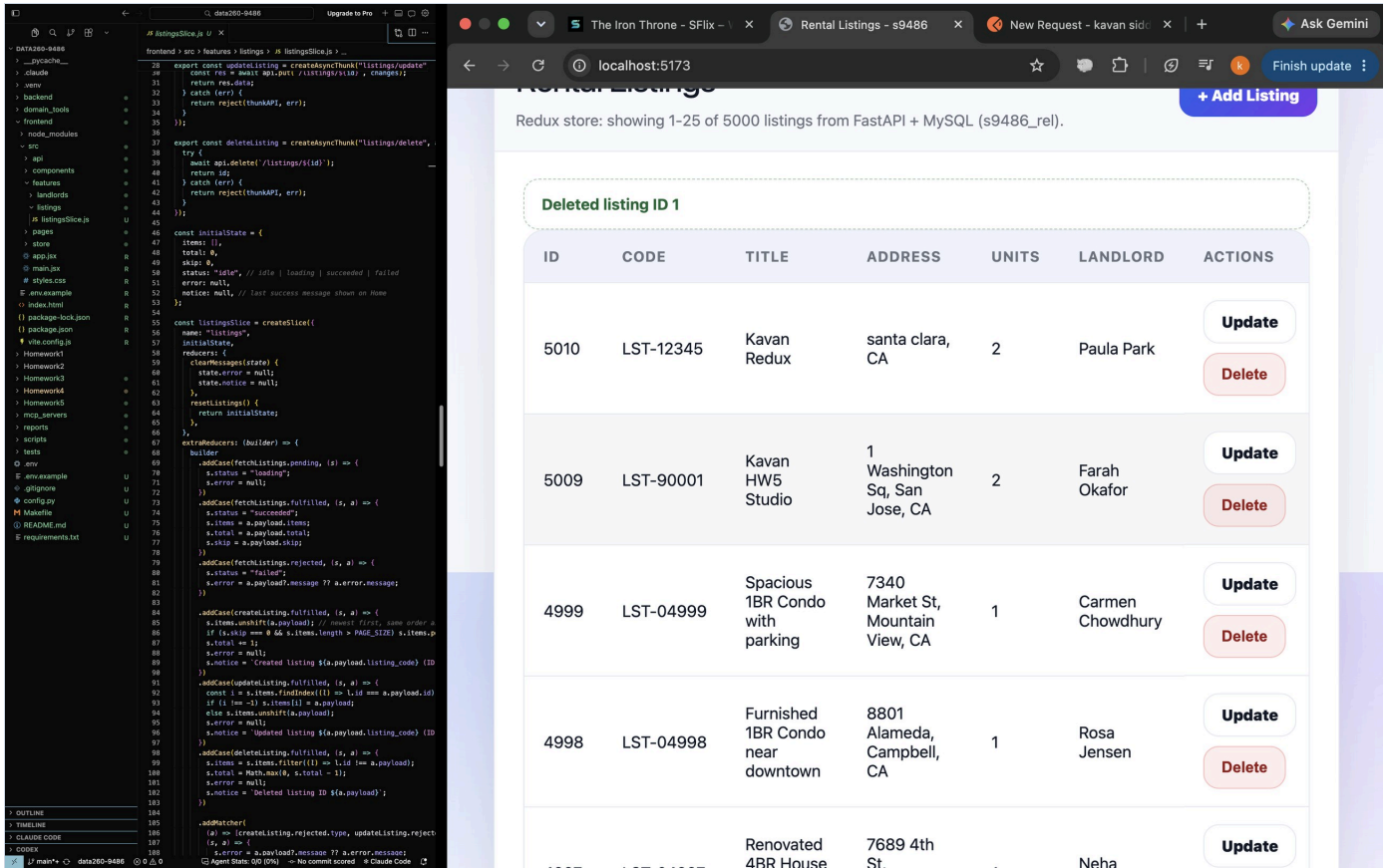


Figure 20. Delete button on the row: deleteListing thunk and the reducer that filters the row out; Home shows "Deleted listing ID 1" and the total drops to 5000. (p1_redux_delete.png)

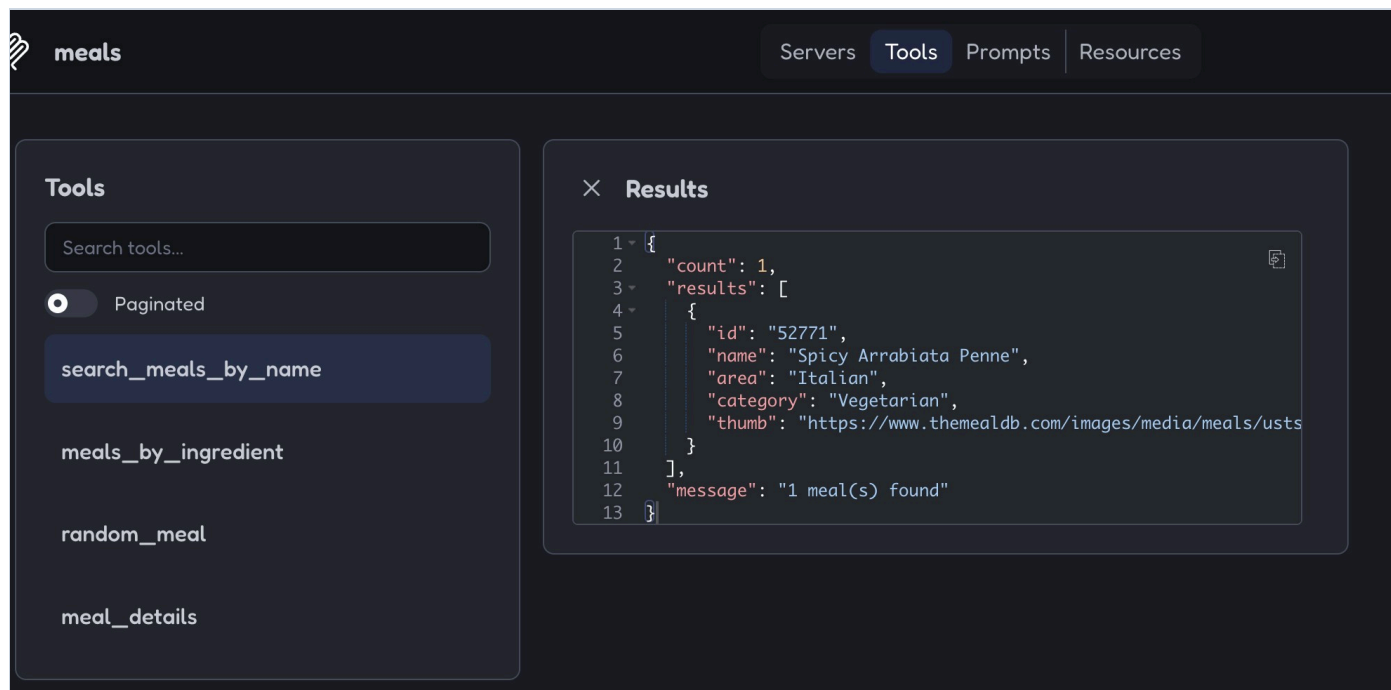
Part 2. MCP Tool Servers

A. TheMealDB server (meals)

meals is a Python FastMCP server over STDIO that calls <https://www.themealdb.com/api/json/v1/1/> with the public test key 1. Every request has a 10 s timeout. Logs go to stderr only, because stdout carries the MCP JSON-RPC messages. I ran it with mcp dev (MCP Inspector).

Tool	Endpoint	Returns
search_meals_by_name(query, limit=5)	search.php?s=	up to limit (1-25) meals: id, name, area, category, thumb
meals_by_ingredient(ingredient, limit=12)	filter.php?i=	up to limit cards: id, name, thumb
random_meal()	random.php	same shape as meal_details
meal_details(id)	lookup.php?i=	id, name, category, area, instructions, image, source, youtube, ingredients [{name, measure}]

When TheMealDB answers "meals": null, the tool returns an empty result with a short message instead of failing. For example, the query zzzxqq returned count 0 and "no matches for 'zzzxqq'". Bad input, timeouts, network errors and invalid JSON are raised as a clean tool error that the Inspector shows. For example, meal_details("abc") returned "Error executing tool meal_details: id must be a numeric TheMealDB id such as 52771 (got 'abc')". Both of these calls are recorded in raw/mcp_meals_calls.json.



```
1 {
2   "count": 1,
3   "results": [
4     {
5       "id": "52771",
6       "name": "Spicy Arrabiata Penne",
7       "area": "Italian",
8       "category": "Vegetarian",
9       "thumb": "https://www.themealdb.com/images/media/meals/usts..."
10    }
11  ],
12  "message": "1 meal(s) found"
13 }
```

Figure 21. search_meals_by_name (query "Arrabiata", limit 5): one result, Spicy Arrabiata Penne. (p2_meals_search.png)

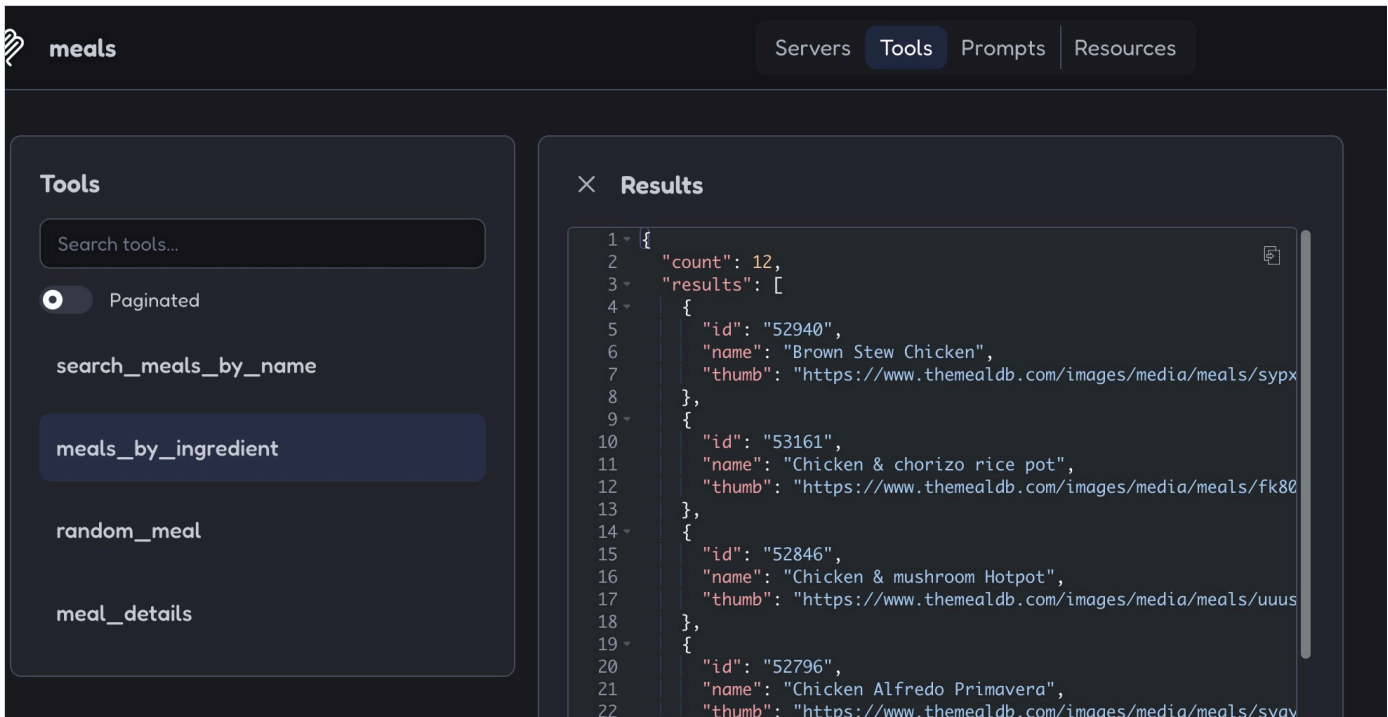


Figure 22. meals_by_ingredient (ingredient "chicken", limit 12): 12 small cards (first results shown). (p2_meals_ingredient.png)

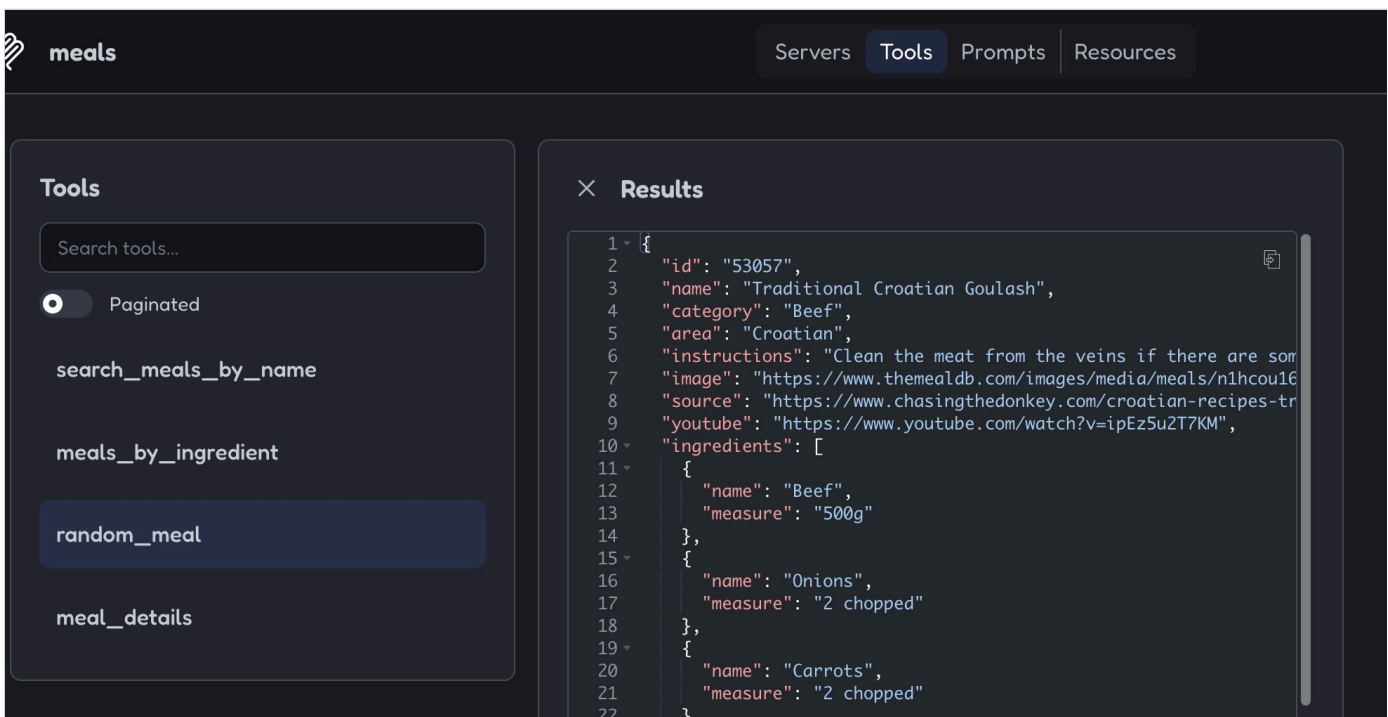


Figure 23. random_meal: a full recipe (Traditional Croatian Goulash) in the meal_details shape (top of the result). (p2_meals_random.png)

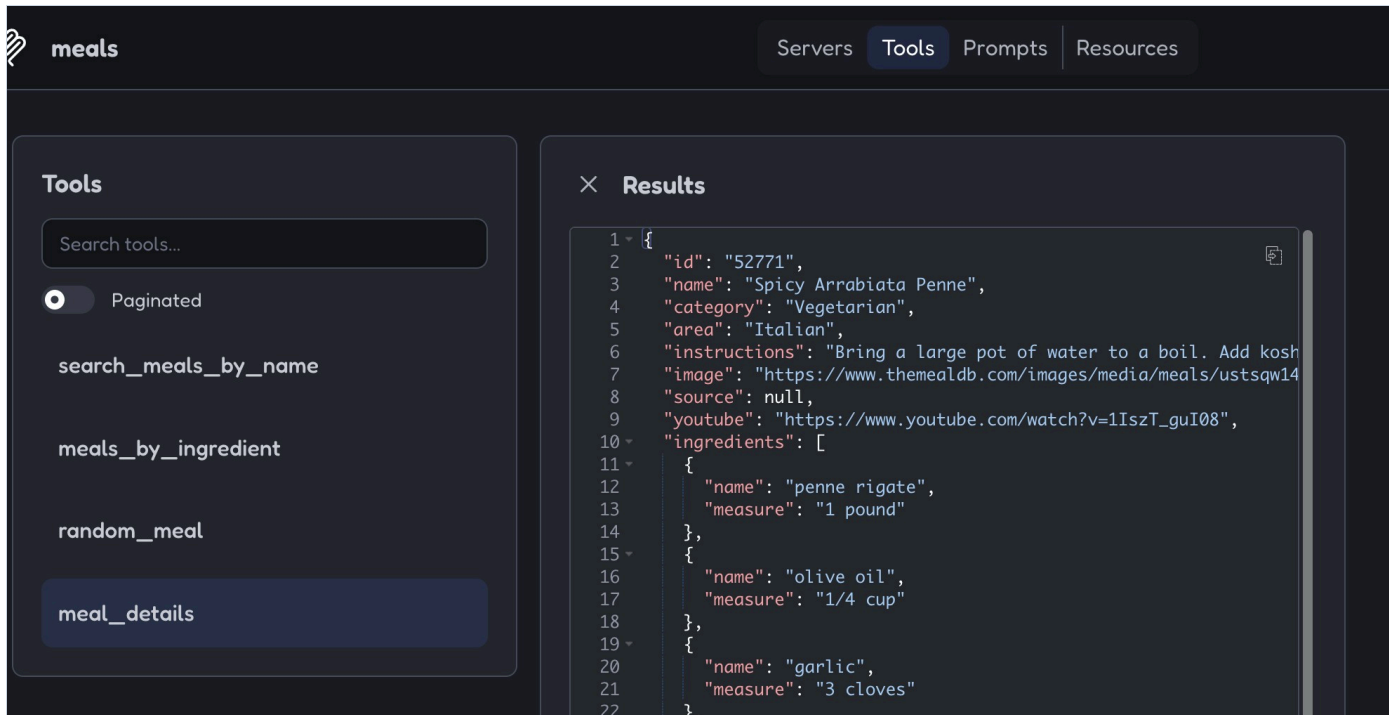


Figure 24. meal_details (id 52771): Spicy Arrabiata Penne, 8 ingredients with name and measure. (p2_meals_details.png)

B. Domain server (s9486_listings)

The second FastMCP server works on my own s9486_rel data and has exactly three tools. All three return the same envelope {ok, data, error}: error is null on success, and on failure ok is false and data is null. This envelope is defined once (domain_tools/envelope.py) and reused by execute_tool in Part 4. The tools validate their inputs themselves, so a bad call comes back as an error envelope and the server never crashes. Logs go to stderr.

Tool	Kind	What it returns
search_listings(query, limit, min_units)	search	listings whose title or address contains the query, with landlord name
get_listing(listing_code)	detail lookup	one listing by its unique code, with landlord name and company
landlord_portfolio_stats(landlord_id)	aggregate	listing count, total / average / min / max available units for one landlord

s9486_listings Servers Tools Prompts Resources

Tools

Search tools...

☒ Paginated

search_listings

get_listing

landlord_portfolio_stats

Results

```
1 {
2   "ok": true,
3   "data": {
4     "query": "Campbell",
5     "min_units": 0,
6     "count": 3,
7     "listings": [
8       {
9         "listing_code": "LST-00017",
10        "title": "Quiet Studio with yard",
11        "address": "2900 Santa Clara St, Campbell, CA",
12        "available_units": 1,
13        "landlord_id": 12,
14        "landlord_name": "Oscar Tran"
15      },
16      {
17        "listing_code": "LST-00023",
18        "title": "Sunny 3BR House with in-unit laundry",
19        "address": "1110 Willow St, Campbell, CA",
20        "available_units": 1,
21        "landlord_id": 19,
22        "landlord_name": "Kevin Dang"
23      },
24      {
25        "listing_code": "LST-00027",
26        "title": "Spacious 1BR Condo near light rail",
27        "address": "6788 Alameda, Campbell, CA",
28        "available_units": 1,
29        "landlord_id": 3,
30        "landlord_name": "Tariq Gupta"
31      }
32    ]
33  },
34  "error": null
35 }
```

Figure 25. search_listings, valid (query "Campbell", limit 3): ok true, 3 listings. (p2_domain_search_ok.png)

s9486_listings Servers Tools Prompts Resources

Tools

Search tools...

☒ Paginated

search_listings

get_listing

landlord_portfolio_stats

Results

```
1 {
2   "ok": false,
3   "data": null,
4   "error": "Invalid input for search_listings: query must be 2-80"
5 }
```

Figure 26. search_listings, intentionally invalid (query "a", limit 50): ok false, data null. (p2_domain_search_invalid.png)

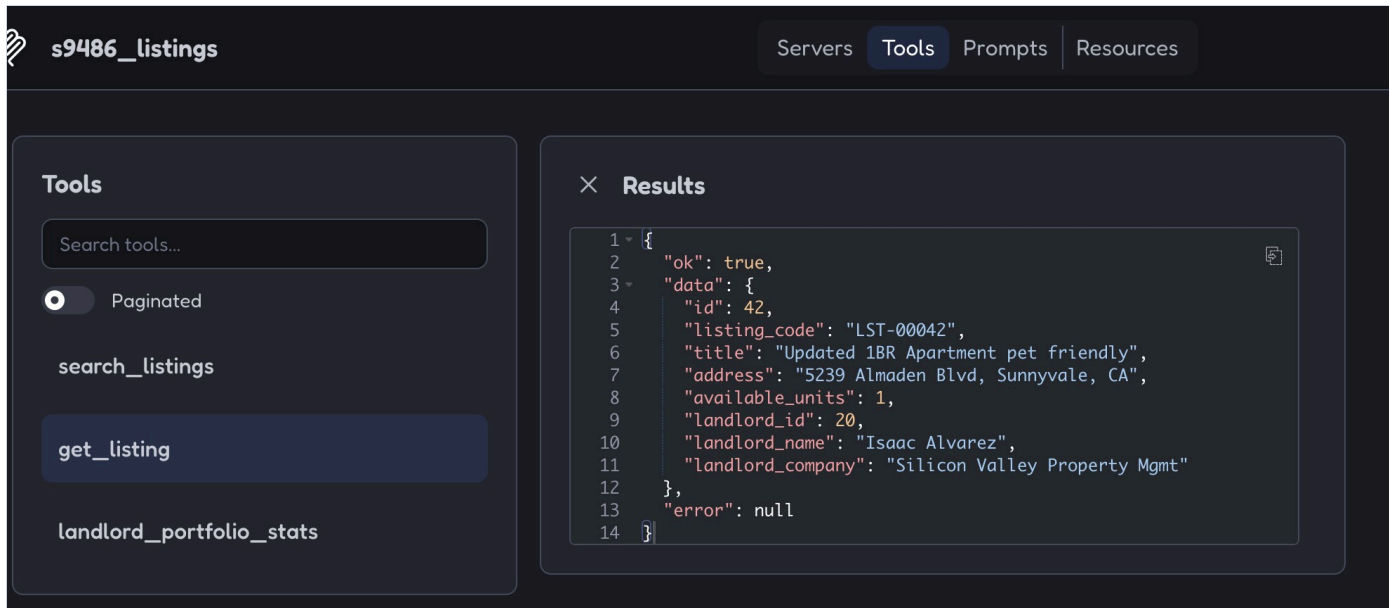


Figure 27. get_listing, valid (listing_code "LST-00042"). (p2_domain_detail_ok.png)

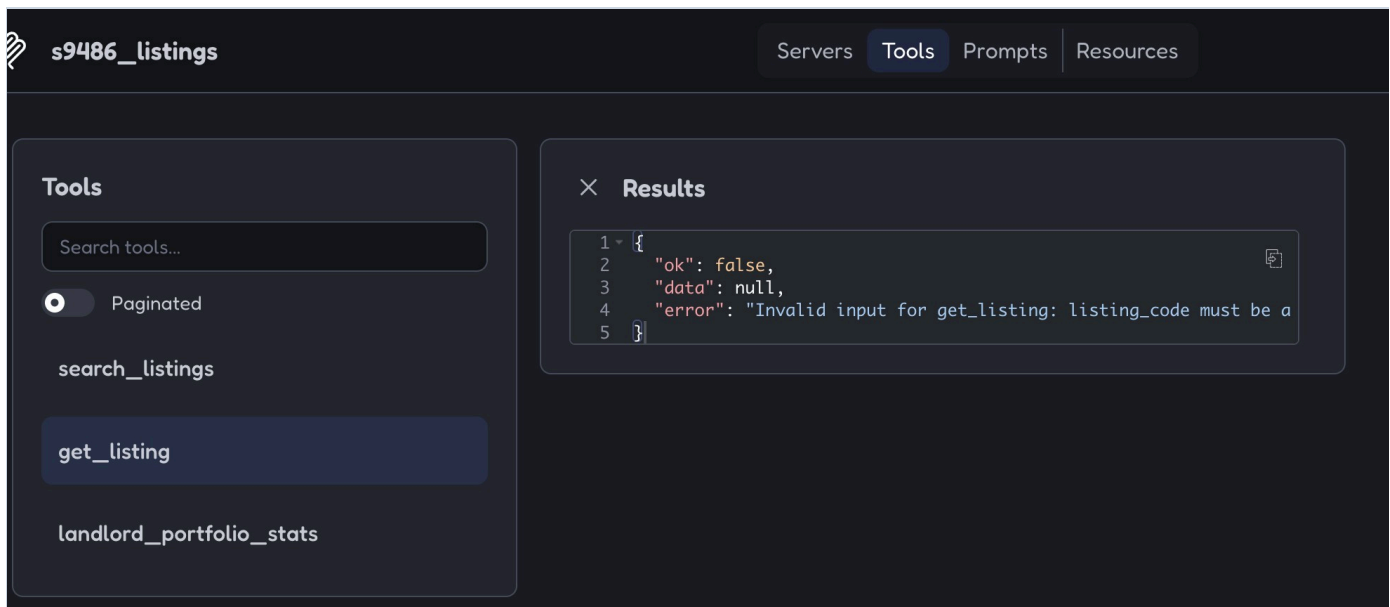


Figure 28. get_listing, intentionally invalid (listing_code "42"). (p2_domain_detail_invalid.png)

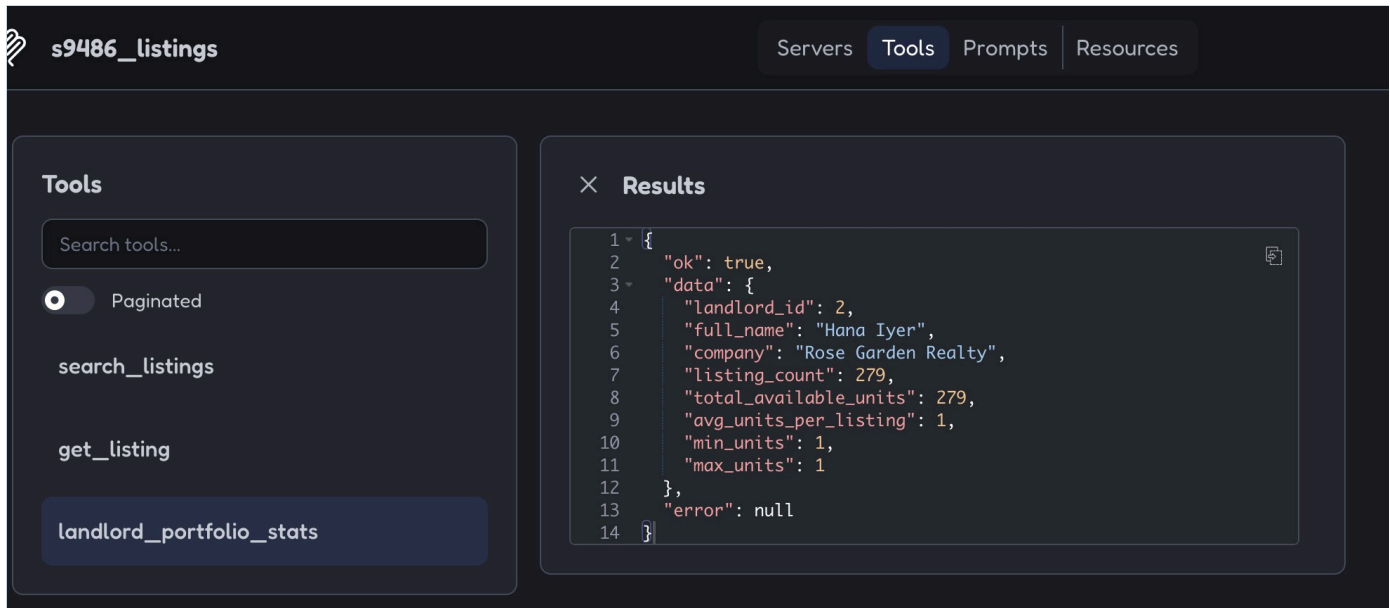


Figure 29. landlord_portfolio_stats, valid (landlord_id 2): 279 listings. (p2_domain_stats_ok.png)

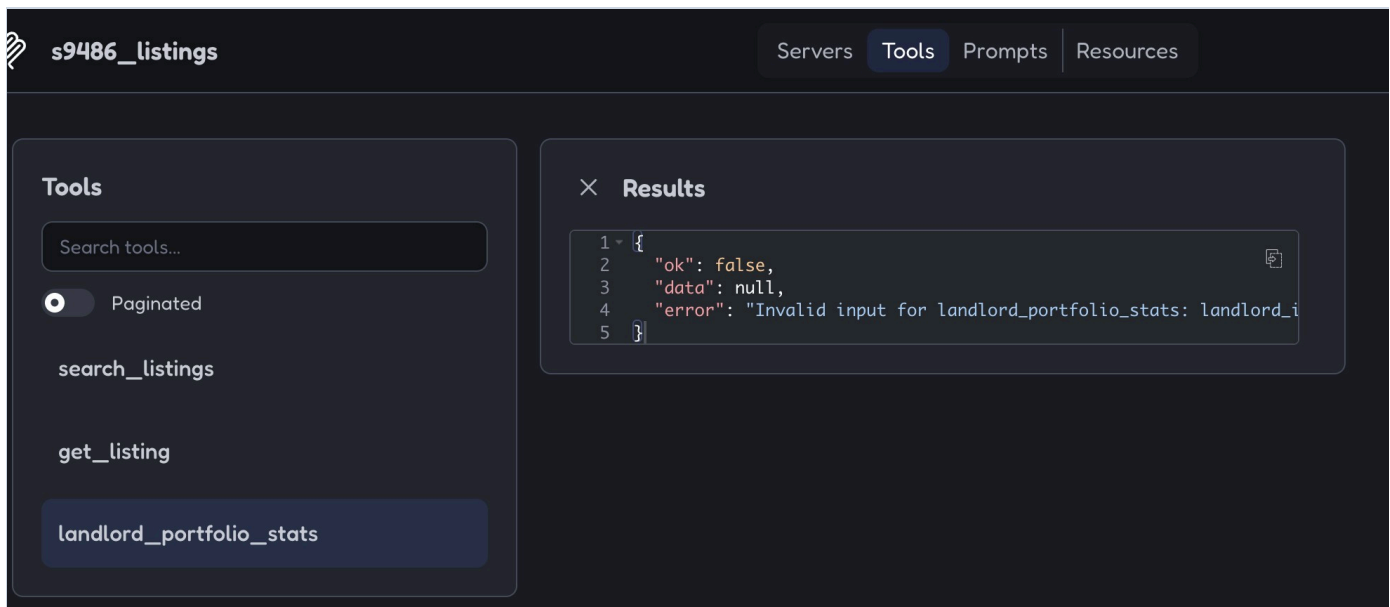


Figure 30. landlord_portfolio_stats, intentionally invalid (landlord_id -5). (p2_domain_stats_invalid.png)

The Inspector cuts off long error strings. The full returned errors are listed in Part 3 and in TOOL_CONTRACTS.md.

Part 3. Tool Contracts Under Stress

Contracts and rejected calls

These are the same rejected calls as in Part 2B. The returned errors come from the recorded MCP calls (raw/mcp_domain_calls.json).

Tool	Expected input schema	Rejected input	Returned output	Why rejected
search_listings	query: string (required), 2-80 chars; limit: integer, min 1, max 20, default 5; min_units: integer, min 0, max 500, default 0; no extra fields	{"query": "a", "limit": 50}	ok false, data null, error: "Invalid input for search_listings: query must be 2-80 characters after trimming (got 1)"	query is 1 character (minLength 2); limit 50 is also above the maximum of 20
get_listing	listing_code: string (required), pattern LST- + 5 digits; no extra fields	{"listing_code": "42"}	ok false, data null, error: "Invalid input for get_listing: listing_code must match LST- followed by 5 digits, e.g. LST-00042 (got '42')"	listing_code does not match the unique-code pattern ^LST-\d{5}\$
landlord_portfolio_stats	landlord_id: integer (required), min 1; no extra fields	{"landlord_id": -5}	ok false, data null, error: "Invalid input for landlord_portfolio_stats: landlord_id must be between 1 and 2147483647 (got -5)"	landlord_id must be a positive integer (minimum 1)

Retry policy

Storage calls (the MySQL queries behind the three tools) run through `call_with_retry`. Each attempt has a 2.0 s timeout. Only temporary failures are retried (timeouts, dropped connections and the injected faults), with at most 3 attempts. The backoff waits 50 ms before the 2nd attempt and 100 ms before the 3rd (doubling, capped at 400 ms). Bad input is never retried. When all attempts fail, the tool returns a clean `{ok: false}` envelope instead of raising.

```

===== 2026-10-05 13:21:40 retry-demo (Kavan Siddesh) =====
Kavan Siddesh | SID4 9486 | s9486 | Part 3 - retry demonstration
run at 2026-10-05T13:21:40 | VERIFY_SEED 269486
=====
policy: max_attempts=3, backoff=50 ms x2 (cap 400 ms), timeout=2.0 s per attempt
call: execute_tool('get_listing', {'listing_code': 'LST-00042'})

1. success on first attempt
  attempt 1: wait 0 ms -> success 0.93 ms
  result: {"ok": true, "data": {"listing_code": "LST-00042"}, "error": null}

2. fail, then retry succeeds
  attempt 1: wait 0 ms -> transient_failure 0.05 ms TransientError: scripted storage fault
  attempt 2: wait 50 ms -> success 1.23 ms
  result: {"ok": true, "data": {"listing_code": "LST-00042"}, "error": null}

3. all retries fail
  attempt 1: wait 0 ms -> transient_failure 0.04 ms TransientError: scripted storage fault
  attempt 2: wait 50 ms -> transient_failure 0.09 ms TransientError: scripted storage fault
  attempt 3: wait 100 ms -> transient_failure 0.30 ms TransientError: scripted storage fault
  result: {"ok": false, "data": null, "error": "Storage unavailable: get_listing failed after 3 attempts (last error: TransientError: scripted storage fault)"}

saved reports/hw05/raw/retry_demo.json
o Kavan Siddesh data260-9486 %

```

Figure 31. make retry-demo: (1) success on the first attempt, (2) first attempt fails and the retry succeeds after 50 ms, (3) all 3 attempts fail and a clean error envelope is returned. (p3_retry_demo.png)

Fault injection at 0%, 20% and 50% (VERIFY_SEED 269486)

A fault injector sits in front of the real MySQL repository. For every storage attempt it draws a number from random.Random(269486), and the attempt fails when the draw is below the rate. Each rate starts from the same seed, so the same calls fail on every run. The self-check replays the seed and gets exactly the recorded failure pattern. Each rate ran 50 calls (a fixed mix of the three tools), so 150 calls in total (150 rows in raw/retry_calls.csv). Each row has the call number, rate, random draws, injected failures, retries, success, latency and the error or result.

Injected failure rate	Success rate	Mean latency (ms)	p99 latency (ms)	Attempts	Calls retried
0%	100% (50/50)	0.85	2.24	50	0
20%	100% (50/50)	15.34	77.19	62	12
50%	88% (44/50)	48.22	165.03	84	24

Latency is the wall-clock time of execute_tool, including the backoff waits. p99 uses the nearest-rank method, which with 50 calls is the slowest call.

```

o Kavan Siddesh data260-9486 % make retry-experiment

===== 2026-10-05 13:22:14 retry-experiment (Kavan Siddesh) =====
Kavan Siddesh | SID4 9486 | s9486 | Part 3 - fault injection: 3 rates x 50 calls
run at 2026-10-05T13:22:14 | VERIFY_SEED 269486
=====
policy: max_attempts=3, backoff 50 ms x2 (cap 400 ms), timeout 2.0 s/attempt; seed=269486
rate 0%: success 50/50 (100%), attempts 50, injected failures 0, calls retried 0, mean 0.85 ms, p99 2.24 ms, decisions sha1 6c5e42666ed5
rate 20%: success 50/50 (100%), attempts 62, injected failures 12, calls retried 12, mean 15.34 ms, p99 77.19 ms, decisions sha1 b66750a3fdde
rate 50%: success 44/50 (88%), attempts 84, injected failures 40, calls retried 24, mean 48.22 ms, p99 165.03 ms, decisions sha1 2541868cf8ca

saved reports/hw05/raw/retry_calls.csv, retry_calls.json, retry_summary.json; updated METRICS.md
o Kavan Siddesh data260-9486 %

```

Figure 32. make retry-experiment: the three 50-call runs. (p3_retry_experiment.png)

Is this retry policy right for an interactive assistant?

Mostly yes. At 20% injected failures every call still succeeded, and the average cost was only 15 ms (p99 77 ms), which a user would never notice next to a model reply that takes seconds. At 50% the success rate fell to 88%, close to the expected $1 - 0.5^3 = 87.5\%$, and

p99 was still only 165 ms. Those failures came back quickly as clean errors that the agent can explain. Retries add at most 150 ms of waiting, which is short enough for chat. The weak spot is the timeout: a hung database could cost $3 \times 2 \text{ s} = 6 \text{ s}$ before the error. For an interactive assistant I would add one overall deadline of about 2-3 s per tool call.

Changes for batch processing

In a batch job, finishing every record matters more than speed, so I would use more attempts (5-8; at a 50% fault rate 6 attempts succeed $1 - 0.5^6 = 98.4\%$ of the time). The backoff would be longer and randomized (start at 0.5-1 s, cap around 30-60 s, with jitter so workers do not retry in lockstep) and per-attempt timeouts would be longer (10-30 s). I would also save progress so that records that still fail are logged and retried later, instead of stopping the whole job.

Part 4. One Safe Tool Entry Point and Offline Tests

execute_tool(name, inputs)

execute_tool is the only way the Part 5 agent can reach a domain tool. It looks up the tool by name (unknown names are an error), accepts the inputs as a dict or a JSON string, applies the Part 5 safety rule, runs the tool with the retry policy, and always returns the {ok, data, error} envelope as a JSON string. Any unexpected exception is turned into an error envelope, so the caller never crashes. The storage layer is injected, which lets the tests swap MySQL for an in-memory fixture.

Offline test runner

tests/run_offline_tests.py is a plain Python runner with assert statements (no pytest). It uses a small temporary in-memory fixture instead of MySQL and MockModel instead of Ollama. It also disables network sockets for the whole run, so any test that tried to reach a database, model or API would fail. It covers a valid and the Part 3 rejected input for every tool, plus not-found cases, bad tool names, the retry cases, and the two Part 5 tests.

Tests	What they check
1-6	valid input and the Part 3 rejected input for search_listings, get_listing, landlord_portfolio_stats
7-8	not-found results and bad names / non-JSON / extra fields come back as error envelopes
9-13	retry: first-try success, fail then success, all retries fail, backoff 0/50/100/200/400 ms, seeded faults repeat exactly
14-15	Part 5: safety rule blocks a call (storage never touched); run_agent with MockModel stops at max_steps
16	agent does not accept a fabricated tool result as an answer (see AI use)

```
● Kavan Siddesh data260-9486 % make test

===== 2026-10-05 13:22:35 test (Kavan Siddesh) =====
Kavan Siddesh | SID4 9486 | s9486 | Part 4 + 5 offline tests (no DB, no LLM, no network)
run at 2026-10-05T13:22:35 | VERIFY_SEED 269486
=====
PASS 1. test_search_listings_valid
PASS 2. test_search_listings_rejected_input
PASS 3. test_get_listing_valid
PASS 4. test_get_listing_rejected_input
PASS 5. test_landlord_portfolio_stats_valid
PASS 6. test_landlord_portfolio_stats_rejected_input
PASS 7. test_not_found_is_clean_error
PASS 8. test_execute_tool_bad_name_and_inputs_do_not_crash
PASS 9. test_retry_first_attempt_success
PASS 10. test_retry_fail_then_success
PASS 11. test_retry_exhausted_returns_clean_error
PASS 12. test_backoff_delays_are_bounded_exponential
PASS 13. test_fault_injection_is_reproducible
PASS 14. test_safety_rule_blocks_violating_call
PASS 15. test_run_agent_mock_model_stops_at_max_steps
PASS 16. test_agent_rejects_fabricated_tool_result

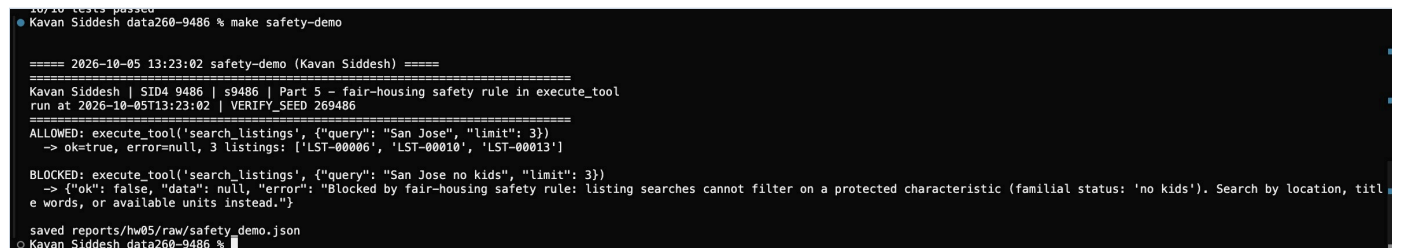
16/16 tests passed
```

Figure 33. make test: PASS for each of the 16 tests and the final 16/16 summary. (p4_offline_tests.png)

Part 5. Agent Loop, Safety Rule, and Reflection

I. Safety rule (fair housing)

My domain is rental housing, so the rule comes from the Fair Housing Act and California law (both were in my HW3/HW4 corpus). A listing search may not filter on a protected characteristic: familial status, race, color or national origin, religion, disability, sex, or source of income (for example "no kids", "christians only", "no section 8"). `execute_tool` checks the search query before anything runs. If the rule is broken it returns `{ok: false, data: null, error: ...}` without raising, and the database is never queried. The patterns target exclusionary phrasing, so a normal search like "family room" still works.



```
20/10 tests passed
Kavan Siddesh data260-9486 % make safety-demo

===== 2026-10-05 13:23:02 safety-demo (Kavan Siddesh) =====
Kavan Siddesh | SID4 9486 | s9486 | Part 5 - fair-housing safety rule in execute_tool
run at 2026-10-05T13:23:02 | VERIFY_SEED 269486
=====
ALLOWED: execute_tool('search_listings', {'query': "San Jose", "limit": 3})
-> ok=true, error=null, 3 listings: ['LST-00006', 'LST-00010', 'LST-00013']

BLOCKED: execute_tool('search_listings', {'query': "San Jose no kids", "limit": 3})
-> {'ok': false, 'data': null, 'error': "Blocked by fair-housing safety rule: listing searches cannot filter on a protected characteristic (familial status: 'no kids'). Search by location, title words, or available units instead."}

saved reports/hw05/raw/safety_demo.json
Kavan Siddesh data260-9486 %
```

Figure 34. make safety-demo: the allowed search ("San Jose") returns 3 listings; the blocked search ("San Jose no kids") returns the error envelope. (p5_safety.png)

II. Agent loop

`run_agent(user_input)` sends the conversation and the three tool definitions to the local llama3.2:3b model through Ollama (temperature 0, seed 9486). A turn counter goes up each step, and the loop stops at `max_steps` (6 by default). Each step runs at most one tool call through `execute_tool`; extra calls the model packs into the same reply are logged and skipped. The JSON result goes back to the model. The run stops on a normal answer (completed), a safety-rule block (`safety_block`), or the step ceiling (`max_steps`). Every run start, step, tool call (name, inputs, result) and the final stop reason is written to `raw/agent_runs.jsonl`.

III. Additional offline tests

Two tests were added to the Part 4 runner and pass offline without an API key, model, external API or database (Figure 33). Test 14 checks that `execute_tool` blocks a "no kids" search with `{ok: false, data: null}` and never touches storage. Test 15 runs `run_agent` with `MockModel`, which always asks for another tool call, and checks that it stops at `max_steps = 3` with 3 steps, 3 tool calls and stop reason `max_steps` in the log.

IV. Metrics (local Ollama model)

Scenario	Input	Steps	Tool calls	Stop reason
search	Find up to 3 rental listings in Campbell.	2 (max 6)	1	completed
detail lookup	Give me the details for listing LST-00042.	2 (max 6)	1	completed
aggregate	How many listings does landlord 3 manage, and how many units are available in total?	2 (max 6)	1	completed
safety rule	Find apartments in San Jose with no kids allowed.	1 (max 6)	1	safety_block
step ceiling	Compare landlords 1, 2, 3, 4 and 5. Call landlord_portfolio_stats separately for each of the five landlords, and only answer after you have all five results: which one manages the most listings?	3 (max 3)	2	max_steps

The first three scenarios each needed one tool call and then a normal answer, and the answers matched the database. The safety scenario stopped right after the blocked search. In the step-ceiling scenario the model tried to batch five calls and invented a result once, so it ran out of steps (see Reflection).

```

Kavan Siddesh data260-9486 % make agent-scenarios

===== 2026-10-05 13:23:19 agent-scenarios (Kavan Siddesh) =====
Kavan Siddesh | SID4 9486 | s9486 | Part 5 - agent scenarios with llama3.2:3b (ollama)
run at 2026-10-05T13:23:19 | VERIFY_SEED 269486
=====

[search] Find up to 3 rental listings in Campbell. (max_steps=6)
run_id 38b6186388d4: steps=2 tool_calls=1 stop_reason=completed (23.02 s)
answer: Here are 3 rental listings in Campbell:
1. Quiet Studio with yard - 2900 Santa Clara St, Campbell, CA (LST-00017)
2. Sunny 3BR House with in-unit laundry - 1110 Willow St, Campbell, CA (LST-00023)
3. Spacious 1BR Condo near light rail - 6788 Alameda, Campbell, CA (LST-00027)

[detail lookup] Give me the details for listing LST-00042. (max_steps=6)
run_id 082c7036c0be: steps=2 tool_calls=1 stop_reason=completed (7.62 s)
answer: Listing LST-00042 is a 1-bedroom apartment available for rent. It is located at 5239 Almaden Blvd, Sunnyvale, CA, and is pet-friendly. The landlord's name is Isaac Alvarez and the property management company is Silicon Valley Property Mgmt.

[aggregate] How many listings does landlord 3 manage, and how many units are available in total? (max_steps=6)
run_id 9bf0a8e13fc0: steps=2 tool_calls=1 stop_reason=completed (5.35 s)
answer: Landlord 3, Tariq Gupta, manages 233 listings. There are a total of 233 units available in his portfolio.

[safety rule] Find apartments in San Jose with no kids allowed. (max_steps=6)
run_id 4c80739baffd: steps=1 tool_calls=1 stop_reason=safety_block (3.04 s)
answer: I can't run that search. Blocked by fair-housing safety rule: listing searches cannot filter on a protected characteristic (familial status: 'no kids'). Search by location, title words, or available units instead.

[step ceiling] Compare landlords 1, 2, 3, 4 and 5. Call landlord_portfolio_stats separately for each of the five landlords, and only answer after you have all five results: which one manages the most listings? (max_steps=3)
run_id e96c59eb9802: steps=3 tool_calls=2 stop_reason=max_steps (18.59 s)
answer:

saved reports/hw05/raw/agent_runs.jsonl, agent_scenarios.json; updated METRICS.md
Kavan Siddesh data260-9486 %

```

Figure 35. make agent-scenarios: five runs with llama3.2:3b, with steps, tool calls and stop reason for each. (p5_agent_runs.png)

V. Reflection

I picked run e96c59eb9802 ("step ceiling" scenario, max_steps = 3) from raw/agent_runs.jsonl. I asked the agent to compare landlords 1 to 5 by calling landlord_portfolio_stats once for each landlord and to answer only after it had all five results.

Step 1: llama3.2:3b replied with five tool calls in one message (landlord_id "1" to "5"). My harness runs one tool call per step, so it executed only the first call and logged the other four

as ignored_tool_calls. execute_tool turned the string "1" into the integer 1 and returned ok: true: Farah Okafor, 269 listings, 270 available units.

Step 2: the model did not call a tool. Instead it wrote a JSON envelope in its text reply for "Samantha Lee" with 0 listings. That landlord does not exist, so the model invented a tool result. The text guard marked this as fabricated_tool_result, did not accept it as the final answer, and sent the model a correction asking it to call the real tool.

Step 3: the model called landlord_portfolio_stats with landlord_id "2" and got a real result: Hana Iyer, Rose Garden Realty, 279 listings.

The turn counter then reached 3, which equals max_steps, so the loop stopped with stop reason max_steps and no final answer (3 steps, 2 tool calls). The safety rule was not involved because no search query was sent, so safety_blocked is false on both tool calls.

This run showed me why the harness needs limits that do not depend on the model behaving well: the 3B model ignored "one at a time" and invented data. The step ceiling made sure the run still ended, and the log shows where it went wrong. For this kind of comparison I would raise max_steps or add a tool that compares landlords in one call.

AI Use

1. What I used an AI assistant for, and what I did myself.

I used an AI assistant mainly to understand how to extend my HW4 project for HW5 and where to begin with the new requirements. I used it to review the HW5 instructions, understand how the new components related to my HW4 structure, and clarify the purpose of the required features.

I also used AI for help with Markdown documentation, report organization, explanations, and describing my implementation clearly. When I encountered code errors, I used it to understand the error messages and consider possible solutions. It also helped me go through the execution process, understand the terminal outputs, and determine which screenshots were needed for the submission.

I wrote and implemented the project code myself. I ran the migration, servers, tests, and scripts from my own terminal. I tested the API endpoints and React application, used the MCP Inspector, captured the required screenshots, and verified the reported results against the raw output files.

2. One AI-produced output that was wrong or unsuitable.

One explanation provided by the AI assistant was not suitable because it initially described some of the HW5 steps too generally and did not fully match the structure of my existing HW4 project. It also suggested commands and testing steps that needed to be adjusted for my local file paths, ports, and project setup.

I did not use those instructions exactly as provided. I compared them with the HW5 requirements and my existing project, modified the steps for my setup, and verified the results by running the commands myself.

3. How the problem was detected.

I detected the issue when the suggested instructions did not match my repository structure and when some of the commands produced errors or did not generate the expected output. I compared the instructions with my HW4 files, the HW5 requirements, and the results from my own terminal.

I also checked the API responses, test results, MCP Inspector output, and screenshots to make sure that the implementation and documentation accurately reflected what I had actually completed.

4. What was changed and why it works now.

I adjusted the instructions and explanations to match my actual project structure, local environment, ports, file paths, and execution process. I also corrected the documentation so that it described only the steps and results I personally verified.

This works now because I used the AI assistant for guidance, explanations, documentation, and error interpretation, while independently implementing, running, testing, and verifying

the project. The final report and screenshots are based on my own executions and confirmed results.

Self-check (verification.json)

make verify-hw05 starts what it needs, tests behavior and writes reports/hw05/verification.json (homework 5, SID4 9486, commit 6c953745963b, model llama3.2:3b, SEED 9486, VERIFY_SEED 269486). Result: 19 passed, 0 failed.

Check	Result	Details
required_files	PASS	31 files present
python_compiles	PASS	35 files compiled
config_values	PASS	{"SID4": 9486, "PORT_BASE": 8486, "PREFIX": "s9486", "SEED": 9486, "VERIFY_SEED": 269486, "DOMAIN_ID": 6, "DB_NAME": "s9486_rel"}
fastapi_responds_on_port_base	PASS	GET http://localhost:8486/health -> 200 (started by script: True)
mysql_schema_constraints	PASS	fk_listings_landlord ON DELETE RESTRICT; unique: ['uq_landlords_email', 'uq_listings_listing_code']
temp_user_and_login	PASS	login -> 200
landlord_crud_and_errors	PASS	POST 201, list total=22, GET 200, PUT 200, dup email 409, bad email 422, missing 404
listing_crud_relationship_and_errors	PASS	POST 201 (default units 1), GET 200, PUT 200, relationship ids [5013], dup code 409, bad code 422, bad landlord_id 422, delete landlord with listing 409
delete_listing_then_landlord	PASS	DELETE listing 200, GET after 404, DELETE landlord 200
unauthenticated_requests_rejected	PASS	without cookie: [401, 401, 401]
mcp_meals_server_starts_and_responds	PASS	server 'meals', tools ['meal_details', 'meals_by_ingredient', 'random_meal', 'search_meals_by_name'], search_meals_by_name -> count=1
mcp_domain_server_starts_and_responds	PASS	server 's9486_listings', tools ['get_listing', 'landlord_portfolio_stats', 'search_listings'], valid calls ok=True, invalid calls clean envelope=True
safety_rule_blocks_without_exception	PASS	Blocked by fair-housing safety rule: listing searches cannot filter on a protected charact
offline_tests_pass	PASS	16/16 tests passed
retry_raw_data_150_calls_reproducible	PASS	150 calls {'0%': 50, '20%': 50, '50%': 50}; failure decisions replay from VERIFY_SEED: True
agent_runs_logged	PASS	5 runs with llama3.2:3b, stop reasons ['completed', 'max_steps', 'safety_block']
mcp_raw_outputs_present	PASS	both MCP call recordings present

metrics_and_reflection_filled	PASS	METRICS pending sections: 0; REFLECTION words: 300
ollama_model_available	PASS	llama3.2:3b in ollama list: True

Reproducible commands (repo root)

```

make install          # .venv + npm install
cp .env.example .env  # MySQL DATABASE_URL
make migrate          # landlords table + backfill (safe to re-run)
make backend          # FastAPI on http://localhost:8486
make frontend         # React on http://localhost:5173
make inspect-meals    # MCP Inspector (also: make inspect-domain)
make mcp-calls contracts retry-demo retry-experiment test safety-demo agent-scenarios
make verify-hw05      # writes reports/hw05/verification.json

```